

Единый документ требований

Наименование проекта

Интеллектуальная система управления оповещениями

Название документа	Единый документ требований к ИТ-решению
Версия документа	1.1
Дата документа	08.08.2026
Назначение документа	Определяет контекст для разработки требований и описывает требования различных уровней, обеспечивая их согласованность и прослеживаемость для успешной реализации проекта ИТ и ЦТ

История изменений

Версия	Дата	Комментарий	Автор
1.0	08.08.2026	Первоначальная версия ЕДТ	
1.1	13.08.2026	Актуализация архитектуры, экономики и требований ИБ	

Содержание

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	5
Введение	6
1. Архитектура решения, используемые технологии и компоненты	7
1.1 Принципы, положенные в основу архитектуры	7
1.2 Представления архитектуры по модели С4	8
1.2.1 Уровень 1: System Context	9
1.2.2 Уровень 2: Containers	10
1.2.3 Уровень 3: Components	12
1.2.4 Динамическое представление	12
1.3 Компоненты решения	13
1.4 Используемые технологии	13
2. Техническая реализация решения	14
2.1 Приём и нормализация событий	14
2.2 Детерминированное ядро корреляции	14
2.3 Маршрутизация и доставка	15
2.4 Веб-интерфейс	15
2.5 Идентификация и ролевая модель	18
2.6 Проверка качества обработки	18
3. Конкурентоспособность и перспективы внедрения	20
3.1 Сравнение с существующими решениями	20
3.2 Преимущества решения	20
3.3 Ограничения пилота	21
3.4 План внедрения	21
4. Использование инструментов искусственного интеллекта	22
4.1 Место ИИ в архитектуре	22
4.2 Реализованные ИИ-сценарии	22
4.3 Соответствие требованиям БТ7	22
4.4 ИИ в процессе разработки	23
5. Инфраструктура решения	24
5.1 Топология развёртывания	24

5.2	Потребление ресурсов	24
5.3	Целевая конфигурация	25
5.4	Развёртывание	26
5.5	Модель масштабирования	27
5.6	Наблюдаемость	27
6.	Информационная безопасность	28
6.1	Подход и границы моделирования	28
6.2	Модель нарушителя	28
6.3	Перечень угроз	29
6.4	Границы доверия	33
6.5	Недоверенный входной контур	33
6.6	Безопасный контур ИИ	34
6.7	Интеграции и административный контур	34
6.8	Доступность критического пути	35
6.9	Данные, секреты и журналы	35
6.10	Сетевая и контейнерная изоляция	36
6.11	Цепочка поставки	37
6.12	Мониторинг безопасности	37
6.13	Ненейтрализованные угрозы и очередность работ	37
6.14	Доказательство эффективности защитных мер	38
7.	Экономика	40
7.1	Методика и вводные данные	40
7.2	Источники измерения	40
7.3	Продуктовые метрики (база и цель)	41
7.4	Собственные допущения	42
7.5	Дерево КПЭ	42
7.6	Затраты	44
7.7	Инвестиционные КПЭ	46
7.8	Чувствительность и риски модели	46
	Заключение	47
	Список использованных источников	48
	Приложение А Матрица трассировки требований на архитектурные решения	50
	Приложение Б Детальная схема архитектуры	53
	Приложение В Схема данных	54

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Сокращения в тексте являются гиперссылками на эту таблицу.

Сокращение	Расшифровка
AP	Архитектурное решение; нумерация AP-01 – AP-17 по матрице трассировки (приложение А)
БТ, ПТ, ФТ, НФТ	Бизнес-требование, процессное, функциональное и нефункциональное требование заказчика; требования к интеллектуальной обработке приведены в таблице 6
API	Application Programming Interface, программный интерфейс приложения
CMDB	Configuration Management Database, база данных конфигурационных единиц (сервисный каталог)
DLQ	Dead Letter Queue, очередь неразбираемых сообщений
DPP	Discounted Payback Period, дисконтированный срок окупаемости
FTE	Full-Time Equivalent, эквивалент полной занятости
GPU	Graphics Processing Unit, графический процессор (используется для инференса модели)
LLM	Large Language Model, большая языковая модель
NPV	Net Present Value, чистая приведённая стоимость
PI	Profitability Index, индекс доходности
SLA	Service Level Agreement, соглашение об уровне обслуживания
TLS	Transport Layer Security, протокол защиты транспортного уровня
WAF	Web Application Firewall, межсетевой экран уровня приложений
C4	Модель описания архитектуры четырьмя уровнями детализации: контекст, контейнеры, компоненты, код
ИБ	Информационная безопасность
КИИ	Критическая информационная инфраструктура
ПДн	Персональные данные
СУБД	Система управления базами данных

Введение

Дежурные инженеры пилотного периметра Блока разведки и добычи получают поток однотипных оповещений из нескольких независимых систем мониторинга. Оповещения не связаны между собой, дублируются при каждом повторении причины и доставляются всем подписчикам одинаково, независимо от критичности сервиса. Следствие — информационный шум: значимые аварии теряются в потоке, время реакции растёт, а вместе с ним растут и потери от простоя.

Цель работы — спроектировать и продемонстрировать веб-платформу управления оповещениями, снимающую информационный шум и потери от простоя в пилотном периметре, без обработки реальных персональных данных и без зависимости от недоступного в рамках кейса домена Active Directory, с гибкой (не жёстко зашитой) ролевой моделью и классификацией событий.

Для достижения цели решаются задачи:

1. разработка дашборда администратора;
2. разработка личного кабинета сотрудника и администратора;
3. интеграция с TrueConf и системами мониторинга;
4. реализация интеллектуальной обработки алертов (снижение шума, приоритизация, маршрутизация) не менее чем одним рабочим ИИ-сценарием;
5. подготовка экономического обоснования внедрения.

Образ результата — веб-портал с личным кабинетом (список текущих аварий, self-service подписки, история) и админ-панелью (настраиваемые группы пользователей, настраиваемая классификация критичности, правила маршрутизации, near-online дашборд), интегрированный с TrueConf для доставки уведомлений и авторизации без реального AD, принимающий поток синтетических разноформатных событий и обрабатывающий их ИИ-контуром, с приложенным экономическим обоснованием.

Работа оформлена в соответствии с ГОСТ Р 7.32-2017 [1], этапность следует ГОСТ 34.601-90 [2], терминология — ГОСТ Р 59853-2021 [3].

Два ограничения кейса определили ключевые архитектурные решения. Отсутствие доступа к домену AD исключает его из контура аутентификации — эту роль принимает на себя TrueConf. Запрет на обработку реальных персональных данных исключает хранение ФИО, телефонов и адресов — в базе остаются только технические псевдонимизированные идентификаторы, а поток событий формируется генератором синтетических данных.

1. Архитектура решения, используемые технологии и компоненты

1.1 Принципы, положенные в основу архитектуры

Архитектура выстроена вокруг четырёх принципов, каждый из которых является прямым следствием требований заказчика.

1. **Детерминированное ядро, необязательный ИИ.** Приём, регистрация, классификация и доставка алерта выполняются правилами, не зависящими от языковой модели. ИИ-контур подключается к этому пути асинхронно и может быть отключён целиком без остановки обработки.

2. **Конфигурация вместо кода.** Классификация критичности, роли, правила маршрутизации, режимы автоматизации и пороги уверенности хранятся как версионизируемая конфигурация. Добавление роли — это запись в таблице, а не развёртывание новой версии.

3. **Единая внутренняя схема события.** Разноформатные источники приводятся к общей схеме индивидуальными адаптерами. Подключение новой системы мониторинга — это новый адаптер, а не правка ядра.

4. **Асинхронная шина между контурами.** Компоненты обмениваются сообщениями через версионизированные топики Kafka [4], поэтому недоступность потребителя не приводит к потере событий. Выбор журналируемой шины вместо синхронных вызовов между сервисами следует практике построения систем, устойчивых к отказу отдельных потребителей [5].

Соответствие принципов основным критериям кейса приведено в таблице 2.

Таблица 2. Основные критерии кейса и архитектурный ответ на них

Критерий	Архитектурное решение	Где реализовано
Снижение информационного шума	Конечный автомат корреляции: дедупликация повторов, свёртка связанных сигналов, подавление «мигающих» источников; поверх — ИИ-функции карантина и семантической дедупликации	Ядро обработки, ИИ-контур
Отсутствие зависимости от AD	Аутентификация через OAuth 2 TrueConf; AD используется только как номинальный справочник атрибутов	Контур идентификации
Отсутствие реальных ПДн	Хранение только технических псевдонимизированных идентификаторов; синтетический поток событий	Хранилище, генератор событий
Гибкая ролевая модель	Роли — строки в таблице, а не перечисление в коде; права назначаются ролями и группами	Контур идентификации
Настраиваемая классификация	Категории, критичность и правила — версионизируемая конфигурация без пересчёта истории	Конфигурация ядра
Near-online дашборд	Отдельный кэш «живого хвоста» для текущей картины и отдельная проекция в СУБД для истории и аудита	Веб-интерфейс, хранилище
Доставка уведомлений	Маршрутизация по сервисному каталогу и подписке, доставка в TrueConf, фиксация статуса и fallback	Контур уведомлений
Устойчивость к отказу ИИ	Fire-and-forget вызов модели, детерминированный fallback на каждом пути отказа	ИИ-контур

1.2 Представления архитектуры по модели С4

Архитектура описывается по модели С4: последовательными уровнями детализации, каждый из которых отвечает на один вопрос и предназначен своей аудитории. Единая схема, отвечающая на все вопросы сразу, перегружена и не читается, поэтому детальная схема вынесена в приложение Б. Приводятся три уровня С4 и одно дополнительное динамическое представление:

1. **Уровень 1, System Context** — границы системы, пользователи и внешние системы (рисунок 1);

2. **Уровень 2, Containers** — исполняемые единицы: процессы, хранилища и шина, с указанием технологий (рисунок 3);

3. **Уровень 3, Components** — внутреннее устройство контейнера «Платформа» (рисунок 4);

4. **динамическое представление** — путь одного алерта и точки принятия решений (рисунок 5).

Уровень 4 (код) не приводится: его роль выполняют листинги и ссылки на модули репозитория в главе 2. Цветовая нотация — стандартная для С4: тёмно-синий — пользователи, синий — проектируемая система и её контейнеры, голубой — компоненты, серый — внешние системы.

Все представления фиксируют *целевую* архитектуру, а не только текущее состояние пилота, и используют единую нотацию:

— **сплошная рамка и сплошная стрелка** — реализовано и работает на пилотном стенде;

— **штриховая рамка и штриховая стрелка** — целевая архитектура: спроектировано, контракт зафиксирован, реализация вынесена за пределы пилота.

Такое разделение позволяет читать схему как план развития, не выдавая проектные решения за работающий код. Перечень нереализованного с обоснованием приведён в разделе 3.3, а построчное соответствие требованиям к интеллектуальным функциям — в таблице 6.

1.2.1 Уровень 1: System Context

Границы системы и её внешние участники приведены на рисунке 1. Существенны две особенности пилота: TrueConf выступает одновременно каналом доставки и провайдером аутентификации, а Active Directory находится вне контура принятия решений и используется только как справочник атрибутов.

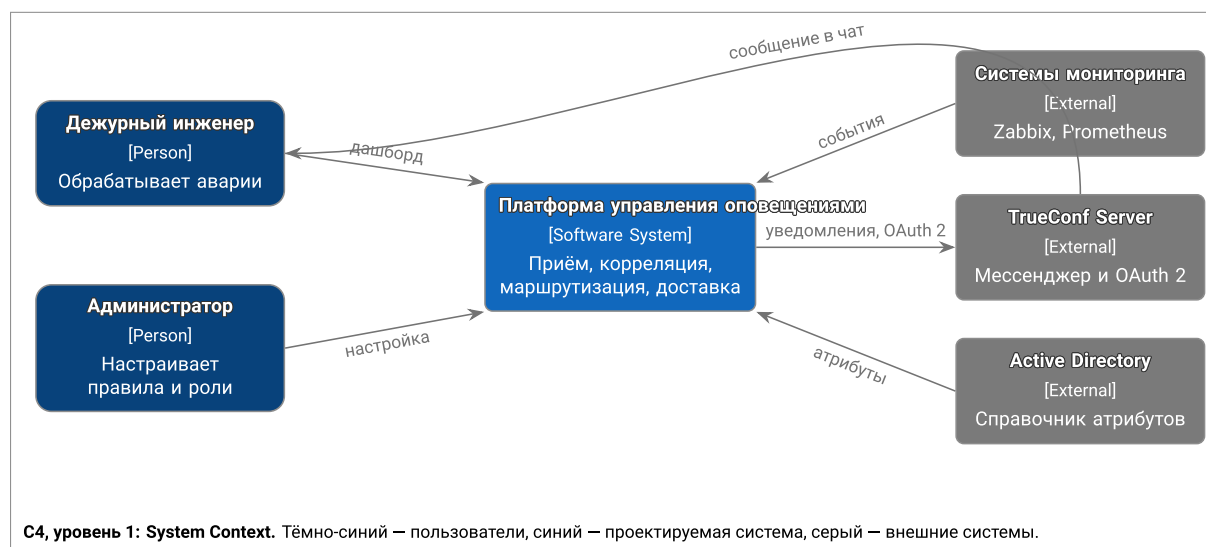


Рисунок 1. С4, уровень 1: контекст системы

Приём событий в целевой архитектуре вынесен в отдельный клиент-коннектор: на каждый узел с системой мониторинга разворачивается свой экземпляр, который приводит формат конкретного вендора к единой внутренней схеме и отправляет результат платформе. Такое размещение решает три задачи: сетевую (система мониторинга не должна иметь маршрута до ядра платформы, достаточно исходящего соединения от коннектора), организационную (подключение новой площадки не требует изменений в платформе) и эксплуатационную (сбой одного коннектора изолирован в пределах своего узла).

Различие текущего и целевого размещения показано на рисунке 2. Оно затрагивает только упаковку и развёртывание: адаптер источника — это реализация одного из двух интерфейсов, и один и тот же код работает в обоих вариантах.

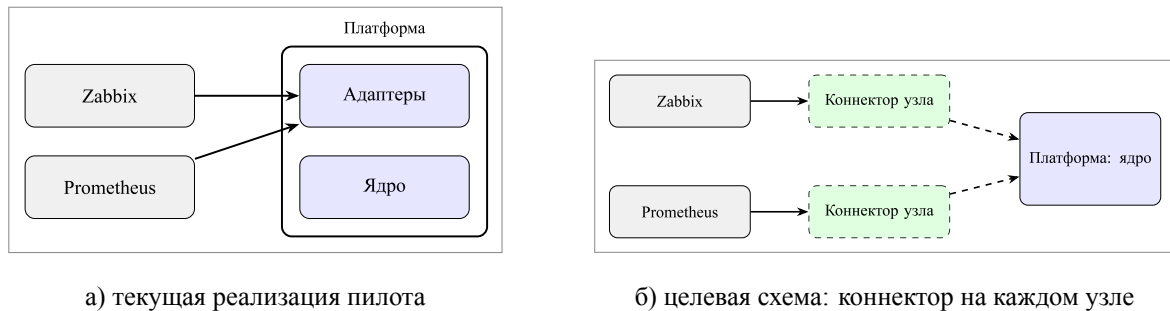


Рисунок 2. Размещение приёма событий: текущая реализация и целевая схема

1.2.2 Уровень 2: Containers

Исполняемые единицы решения приведены на рисунке 3. Каждый контейнер — это отдельный процесс или хранилище, разворачиваемое и масштабируемое независимо; в скобках указана технология реализации. Хранилище и ИИ-обработчик вынесены в сторону от основной цепочки, поскольку не лежат на критическом пути доставки.

Граница «коннектор — платформа» реализована как контракт, а не как способ запуска: адаптер источника — это реализация одного из двух интерфейсов (источник вызывает нас по webhook либо мы опрашиваем источник), поэтому один и тот же адаптер работает и в составе платформы, и как отдельный процесс на узле мониторинга. В текущей реализации пилота адаптеры Alertmanager и Zabbix выполняются внутри процесса платформы — это упрощает стенд, где все системы мониторинга находятся на одном узле; целевая схема развёртывания — отдельный клиент-коннектор на каждый узел, как показано на рисунке 1. Изменение затрагивает только развёртывание: контракты сообщений и код адаптеров остаются прежними.

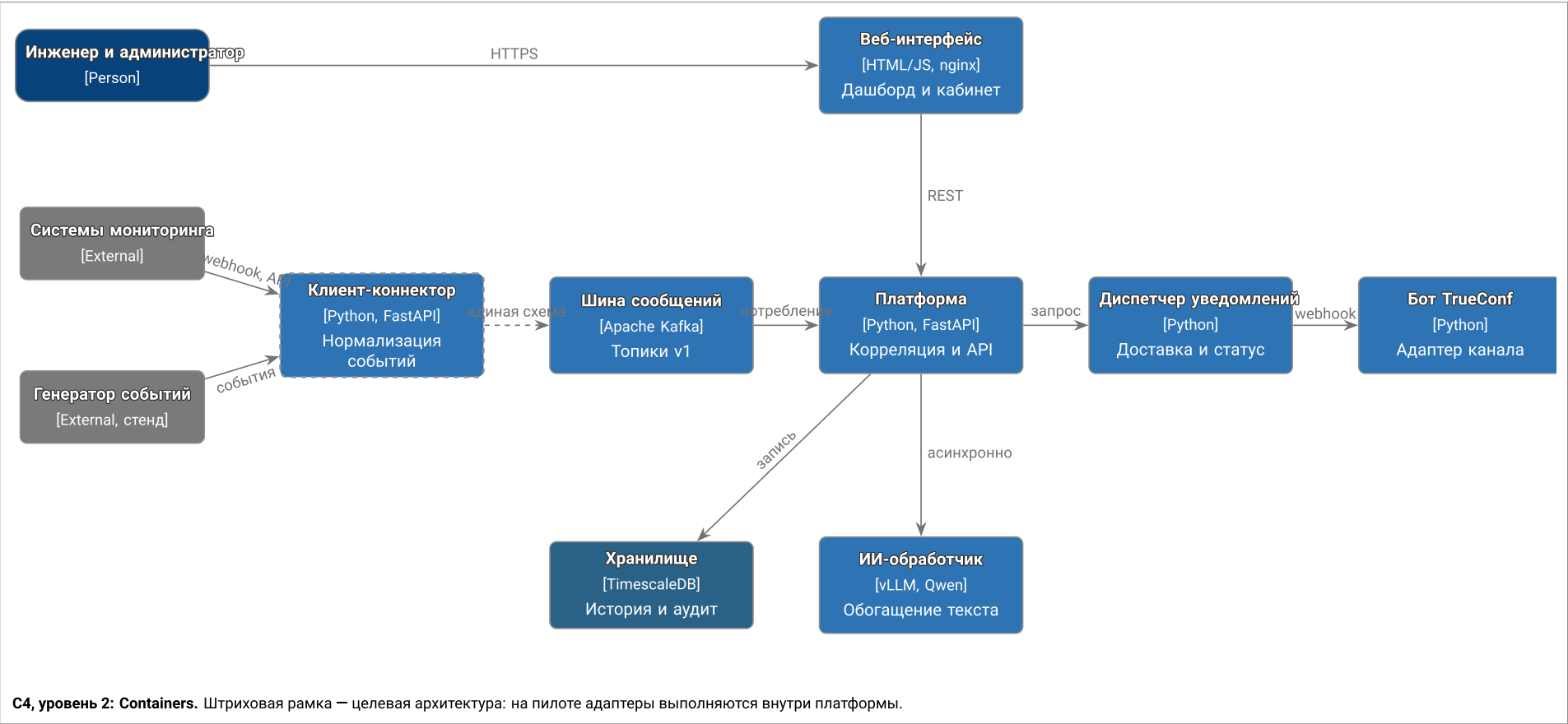


Рисунок 3. С4, уровень 2: контейнеры решения

1.2.3 Уровень 3: Components

Внутреннее устройство контейнера «Платформа» приведено на рисунке 4. Компоненты соответствуют модулям репозитория: приём и нормализация, фильтрация, корреляция, маршрутизация, API запросов, идентификация и клиент ИИ. Клиент ИИ — единственный компонент, обращающийся к модели: второго пути вызова в коде нет, что делает контроль периметра и тайм-аутов проверяемым.

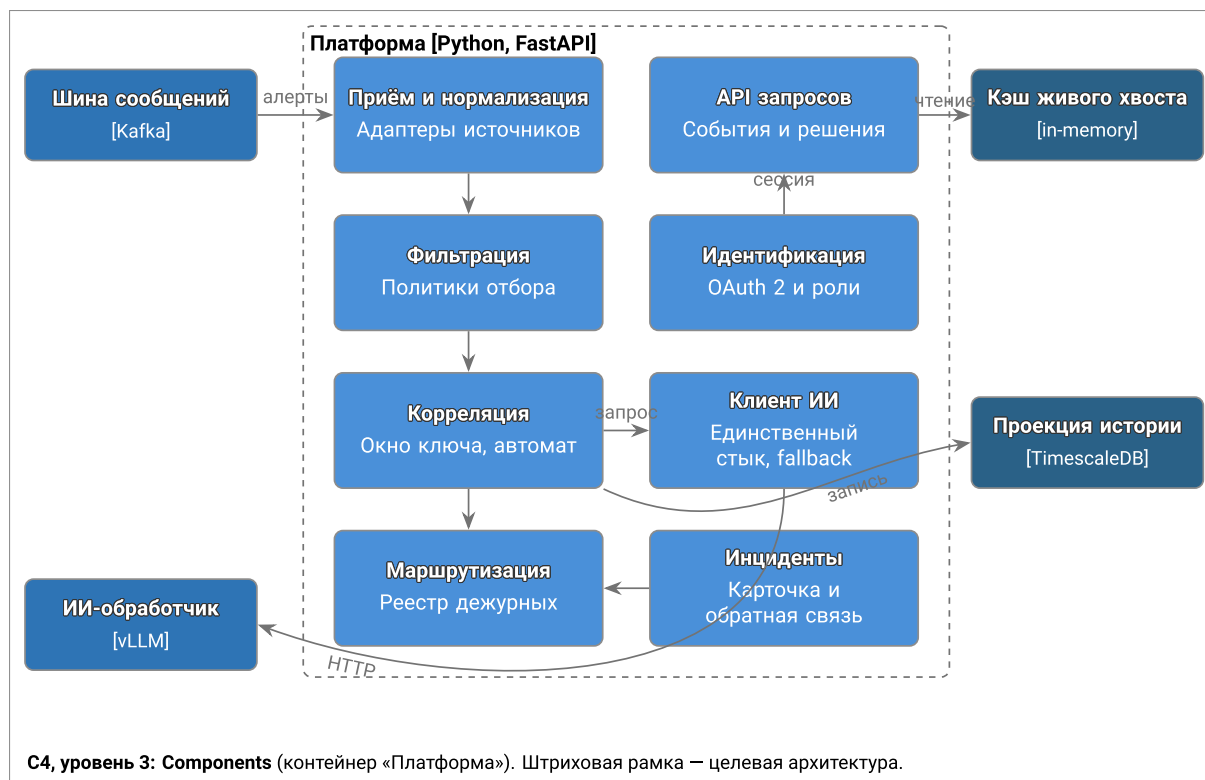


Рисунок 4. С4, уровень 3: компоненты контейнера «Платформа»

1.2.4 Динамическое представление

Путь одного алерта и точки принятия решений приведены на рисунке 5. Пунктиром показан ИИ-контур: он ответвляется от основного пути после того, как детерминированное решение уже принято, и поэтому не может задержать доставку.

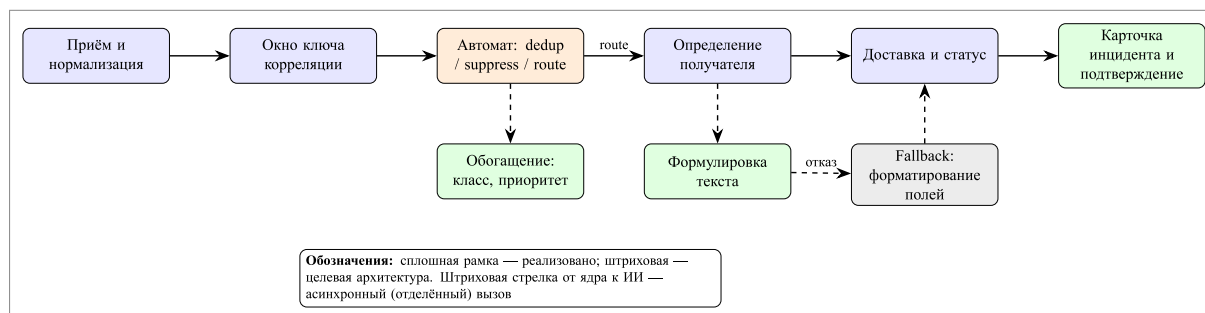


Рисунок 5. Динамическое представление: путь алерта и точки принятия решений

Полная матрица трассировки требований заказчика на архитектурные решения (AP-01 – AP-17) приведена в приложении А.

1.3 Компоненты решения

Решение развёртывается как набор независимых стеков, каждый со своей сетью и томами; зоны ответственности приведены в таблице 3.

Таблица 3. Компоненты решения

Компонент	Зона ответственности	Технологии
Клиент-коннектор	Экземпляр на каждый узел мониторинга: приём webhook или опрос API вендора, приведение к единой схеме	Python, FastAPI
Генератор событий	Синтетический поток разноформатных событий, оценка по триггерным правилам, публикация в шину	Python, Kafka
Шина сообщений	Версионированные топики событий, алертов, решений, ИИ-запросов и результатов; очередь ошибок	Apache Kafka
Платформа (ядро)	Нормализация, фильтрация, корреляция, принятие решений, API для интерфейса, идентификация	Python, FastAPI
ИИ-контур	Семантическая обработка алертов: карантин, приоритизация, саммаризация, объяснение решений	vLLM, Qwen
Хранилище	Гипертаблицы истории событий, алертов и решений ИИ с политиками retention; реляционные таблицы личностей, ролей и связей	TimescaleDB
Веб-интерфейс	Дашборд администратора и личный кабинет; обратный прокси к API платформы	HTML/JS, nginx
Уведомления	Диспетчер доставки: чтение запросов на уведомление, отправка, фиксация статуса	Python, Kafka
Бот TrueConf	Адаптер обязательного канала доставки: приём запроса и отправка сообщения в личный чат	Python, TrueConf Chatbot API
Системы мониторинга	Реальные источники алертов пилота	Zabbix, Prometheus
Каталог AD	Номинальный справочник атрибутов (подразделение, команда) без роли в аутентификации	Samba AD DC

1.4 Используемые технологии

2. Техническая реализация решения

2.1 Приём и нормализация событий

Ингест реализован как точка расширения, а не как ветвление по названию вендора. Определены два интерфейса подключения источника:

а) источник вызывает платформу — webhook по адресу вида `/v1/integrations/<slug>/webhook`; маршрут находит адаптер, зарегистрированный на данный slug, и передаёт ему тело запроса целиком;

а) платформа вызывает источник — адаптер реализуется как асинхронный итератор, который ядро опрашивает самостоятельно.

По умолчанию подключены адаптеры Alertmanager [6] и Zabbix [7]. Формат полезной нагрузки полностью принадлежит адаптеру: Alertmanager присылает массив объектов, Zabbix — плоский объект; маршрут о различии не знает. Некорректная нагрузка приводит к отказу с кодом 422, а попытка зарегистрировать два адаптера на один slug обнаруживается при запуске, а не молча затеняет один другим. Дополнительно по расписанию выполняется сверка открытых проблем через API Zabbix: webhook — быстрый путь, опрос — восстановление пропущенных за время недоступности событий.

Внутренняя схема события намеренно минимальна и открыта: обязательны только источник и статус, время подставляется при отсутствии, а любые дополнительные поля вендора сохраняются как есть. Это позволяет подключить новый источник или добавить производные признаки без миграции схемы. Схема алерта, напротив, строгая: правило, критичность, метрика и порог обязательны, так как на них опирается корреляция.

2.2 Детерминированное ядро корреляции

Каждый принятый алерт проходит потоковую обработку: поэтапное преобразование и последующая свёртка по ключу корреляции в пределах временного окна. Ключ выбирается по первому доступному признаку: явный идентификатор корреляции, затем сервис, затем пара «источник — метрика».

Политика свёртки реализована как конечный автомат на каждый ключ (рисунок 6). Идентичностью сигнала считается тройка «правило — метрика — источник»: два разных сигнала под одним ключом — это дополнительное свидетельство одной аварии, а не повтор.

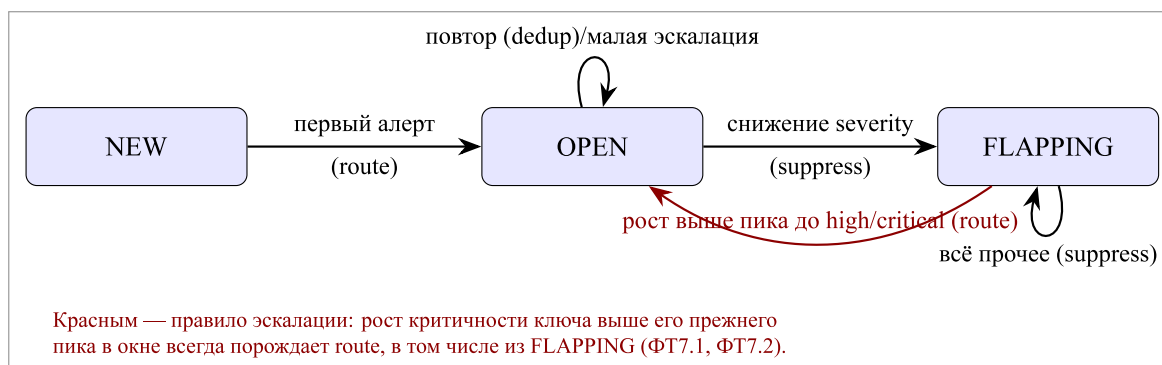


Рисунок 6. Автомат корреляции: состояния ключа и принимаемые решения

Состояние не хранится как изменяемое поле: оно выводится проигрыванием всего окна ключа при каждом вызове. Благодаря этому свёртка остаётся чистой функцией от пары «ключ — окно», а признак нестабильности не может рассинхронизироваться с историей. Неизвестная метка критичности не выводит ключ из обработки, а трактуется как warning — один нестандартный вендорский ярлык не должен ломать обработку.

Каждое принятое решение публикуется в топик решений и доступно через API, то есть является наблюдаемым артефактом, а не внутренним состоянием.

2.3 Маршрутизация и доставка

Решение типа «маршрутизировать» преобразуется в запрос на уведомление: реестр маршрутизации отображает «сервис — команда — дежурный — адрес доставки». Идентификатор инцидента выводится детерминированно из ключа корреляции, поэтому все алерты одной группы получают один идентификатор без хранения состояния — это же обеспечивает дедупликацию на стороне диспетчера после перезапуска.

Существенны два порядковых решения. Во-первых, получатель определяется до обращения к модели: алерт, у которого нет владельца, не должен занимать слот GPU — при лавине событий это разница между очередью, которая разбирается, и очередью, которая не разбирается. Во-вторых, отсутствующая или повреждённая конфигурация маршрутизации деградирует до «никто не уведомлён» с отражением в метрике, а не останавливает приём событий.

Доставка вынесена в отдельный диспетчер: он читает запросы на уведомление, отправляет их и публикует результат — доставлено или не доставлено. Обязательный канал TrueConf реализован как адаптер-бот, принимающий запрос по webhook и отправляющий сообщение в личный чат.

2.4 Веб-интерфейс

Интерфейс разделён на две группы: управление (сводка, пользователи, матрица ответственности, системы, маршрутизация, роли и права, политики SLA, интеграции, аудит) и мониторинг (инциденты, алерты, события, решения корреляции). Он обращается к платформе через обратный прокси, поэтому браузер работает с единственным origin — сессионная cookie аутентификации проходит через прокси без настройки CORS.

Отдельно оговаривается честность отображения: разделы, не подключённые к данным, показываются как макеты с явным баннером «Демонстрационные настройки» (виден на рисунках 7–9), а не как правдоподобные фиктивные данные. Тот же принцип действовал и на уровне API: пока управление инцидентами не было реализовано, соответствующие методы отвечали кодом 501, а не пустым списком, поскольку пустой список неотличим от «сейчас ничего не происходит». Сейчас эти методы работают: инцидент выводится из решений ядра, открывается только маршрутизацией (но не дедупликацией и не подавлением) и подтверждается дежурным.

Главный экран — сводка эксплуатации (рисунок 7): счётчики событий и алертов, распределение по критичности, состояние компонентов платформы и блок «Требуют внимания». Состояние платформы, Kafka, базы идентификаций и проекции алертов читается с backend в реальном времени; операционные показатели на снимке — макет, о чём сообщает баннер.

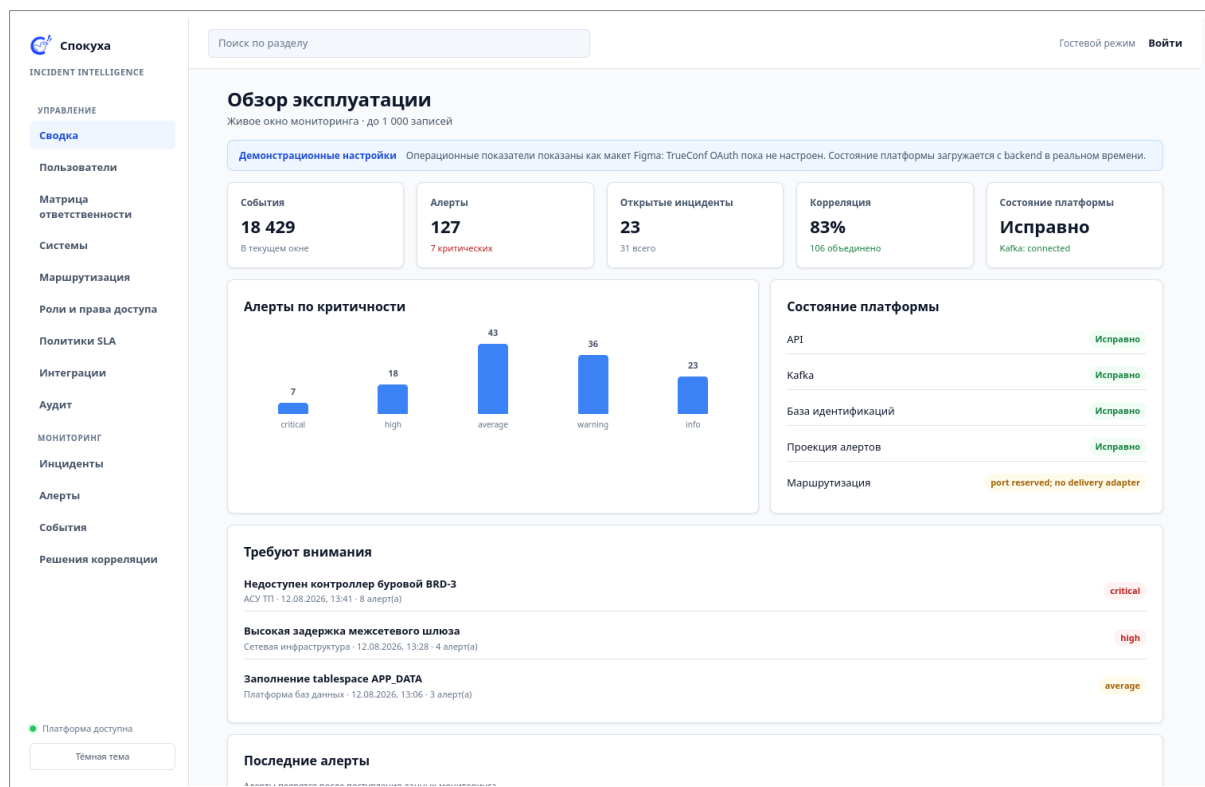


Рисунок 7. Сводка эксплуатации: near-online дашборд

Раздел «Системы» (рисунок 8) объединяет каталог контролируемых систем с их критичностью, владельцами и источником мониторинга и живой перечень контуров платформы с их состоянием — включая honest-статусы вида «port reserved; no delivery adapter» и «deferred; Alertmanager owns alert lifecycle» для того, что ещё не подключено.

Административная часть представлена разделом маршрутизации (рисунок 9): получатели и группы, правило маршрутизации по критичности и источнику, условие отбора, первичный получатель и время эскалации. Экран реализован по макету и подключается к административному API на следующем этапе.

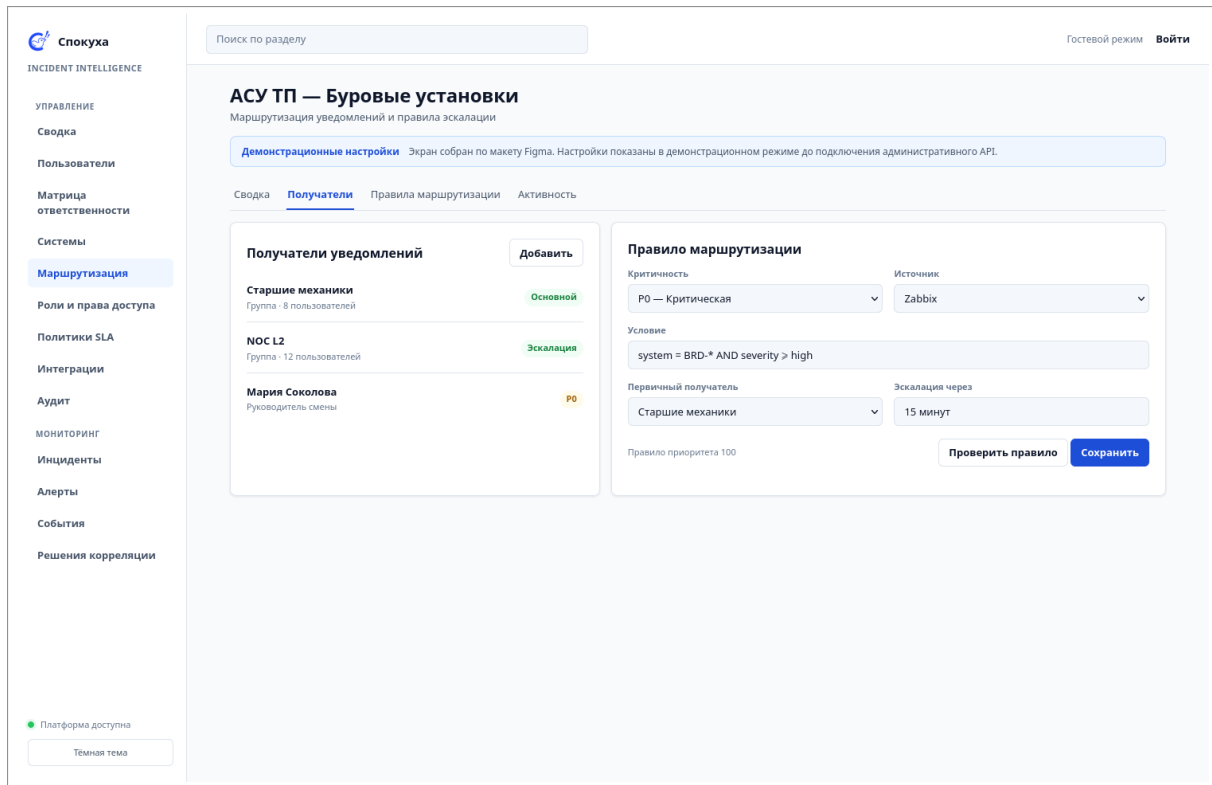


Рисунок 9. Админ-панель: правила маршрутизации и эскалации

Для оперативной картины и для истории используются два разных хранилища: ограниченный кэш «живого хвоста» в памяти отвечает на вопрос «что происходит сейчас» (именно это опрашивает дашборд), а проекция в TimescaleDB [8] — на вопросы истории и аудита. Растягивать кэш на обе задачи было бы ошибкой: требования к ним противоположны.

2.5 Идентификация и ролевая модель

Аутентификация выполняется по OAuth 2 в потоке Authorization Code [9] через TrueConf Server [10]. Роли хранятся в таблице и засеяны значениями «наблюдатель», «инженер», «администратор» — добавление роли является вставкой записи, а не выпуском новой версии кода. Первый вход любой учётной записи автоматически создаёт личность с минимальной ролью, а роль повышает администратор.

Связь с Active Directory реализована как номинальный справочник атрибутов (подразделение, команда) в собственной таблице платформы: платформа никогда не создаёт и не изменяет учётные записи в реальном каталоге. Это прямое следствие ограничения кейса и одновременно снятие зависимости от недоступного домена.

2.6 Проверка качества обработки

Для честной оценки фильтрации подготовлен воспроизводимый корпус событий. Он генерируется декларативно: факты топологии (сервисы, узлы, зависимости, команды, график дежурств, окна обслуживания) отделены от правил вывода (транзитивное распространение отказа, порождение шума, понижение значимости в окне обслуживания, разрешение адресата). Эталонная разметка не проставляется руками, а выводится теми же правилами, поэтому не может разойтись с данными. Корпус проверяется набором инвариантов

до записи: например, смены дежурств обязаны точно покрывать всё время корпуса без пропусков и наложений.

Текущий корпус: 1053 алерта (545 значимых, 508 шумовых), 8 инцидентов, 2 часа модельного времени. Оценка выполняется прогоном корпуса через реальный конвейер в одном процессе — без Kafka, HTTP и развёртывания, за счёт внедряемого источника времени. Результаты приведены в таблице 4.

Таблица 4. Качество обработки на эталонном корпусе (1053 алерта, 8 инцидентов)

Вариант	Уведомлений	Снижение шума	Полнота	Точность
Без фильтрации	1053	0,0 %	8/8	20,5 %
Автомат корреляции	44	95,8 %	8/8	31,8 %

Из 44 уведомлений варианта с корреляцией 14 адресованы верно, 13 ушли не тому дежурному и 17 остались шумовыми (из них лишь одно — работа в окне обслуживания). Для сравнения, базовая линия: 216 верных, 329 не тому адресату и 508 шумовых.

Полнота 8/8 достигнута не ослаблением фильтрации, а отдельным правилом эскалации. Ранее второй инцидент, начавшийся, пока первый ещё каскадирует через те же сервисы, попадал на уже открытый ключ и поглощался им: два одновременных инцидента, делящих сервис, были неразличимы для корреляции по ключу сервиса, и на корпусе терялся один инцидент из восьми. Автомат дополнен правилом: рост критичности до *high* или *critical выше пика, который ключ уже видел в окне*, всегда порождает маршрутизацию — в том числе из состояния нестабильности. Это прямое следствие запрета автоматически скрывать и подавлять критичные алерты (ФТ7.1, ФТ7.2).

Правило остаётся экономным по построению: срабатывание привязано к росту *выше прежнего пика*, поэтому каждый уровень критичности порождает не более одного уведомления на окно, а ключ, уже работающий на *critical*, повторно не уведомляет. Цена восстановленной полноты измерена: восемь дополнительных уведомлений (36 → 44) и 0,8 процентного пункта снижения шума. Точность при этом выросла с 27,8 до 31,8 %.

Оставшееся ограничение точности не устранено: уведомляется дежурный *сигнализирующего*, а не *сломавшегося* сервиса, поэтому 13 из 44 уведомлений уходят не тому человеку. Это устраняется учётом первопричины и данных CMDB.

Следует прямо оговорить границы измерения: один и тот же источник задаёт и шум, и критерий шума, поэтому корпус — честный стенд для сравнения вариантов между собой, но не доказательство эффективности на реальном трафике. Для последнего нужны работающий приём из промышленного Zabbix и канал обратной связи от инженеров.

3. Конкурентоспособность и перспективы внедрения

3.1 Сравнение с существующими решениями

Задача управления оповещениями решается тремя классами продуктов: зарубежными SaaS-платформами реагирования на инциденты, встроенными средствами самих систем мониторинга и российскими ITSM-платформами. Сравнение по значимым для пилота критериям приведено в таблице 5: **зелёным** отмечено соответствие критерию, **жёлтым** — частичное, **красным** — несоответствие.

Таблица 5. Сравнение классов решений

Критерий	Зарубежный SaaS	Средства мониторинга	ITSM-платформы	Настоящее решение
Размещение данных	Внешнее облако	В контуре	В контуре	Полностью в контуре
Доступность в РФ	Ограничена	Да	Да	Да
Несколько источников сразу	Да	Только свой	Частично	Да, адаптерами
Корреляция между источниками	Да	Нет	Ограниченно	Да
ИИ в контуре организации	Обычно внешние API	Нет	Редко	Да, локальный инференс
Доставка в TrueConf	Нет	Нет	Через интеграцию	Штатный канал
Работа без реального AD	Частично	—	Обычно требуется	Да, OAuth 2 TrueConf
Стоимость владения	Подписка на пользователя	Входит в мониторинг	Лицензия и внедрение	Инфраструктура и сопровождение

3.2 Преимущества решения

— Полное размещение в корпоративном контуре, включая инференс языковой модели: исходящий доступ к внешним API моделей не требуется.

— Снижение шума подтверждено измерением на воспроизводимом корпусе, а не заявлено (таблица 4); вместе с ним измеряются и неудобные метрики — полнота и точность адресации.

— Работоспособность при отключённом ИИ: интеллектуальные функции являются надстройкой над детерминированным ядром, а не условием его работы.

— Настраиваемость вместо доработки: классификация, роли, правила маршрутизации и режимы автоматизации — конфигурация.

— Открытая точка расширения для источников: подключение новой системы мониторинга не затрагивает ядро.

3.3 Ограничения пилота

Честная оценка зрелости необходима для планирования внедрения:

- состояние «инцидент решён» не выводится: публикуемого сигнала о восстановлении нет, а вывод его из «перестали приходить алерты» был бы догадкой, выданной за факт;
- часть разделов админ-панели существует в виде макетов и явно помечена как макеты;
- корреляция ведётся по ключу сервиса; одновременные инциденты на общем сервисе разделяются только при росте критичности до high или critical, при равной критичности они по-прежнему сливаются;
- маршрутизация опирается на владельца сигнализирующего сервиса, а не перво-причины;
- масштабирование горизонтально возможно, но требует вынесения состояния ограничителя запросов и кэша во внешнее хранилище.

3.4 План внедрения

Внедрение планируется поэтапно, с управлением через флаги функциональности и задокументированной процедурой отката:

1. подключение первого источника в режиме наблюдения, дублирование критичных уведомлений существующим способом;
2. включение корреляции и маршрутизации на ограниченном наборе сервисов, сверка адресатов с дежурными расписаниями;
3. включение ИИ-функций в режимах shadow и suggest, накопление обратной связи;
4. расширение периметра источников и подписчиков, отключение дублирования;
5. переход отдельных функций в режим ограниченной автоматизации по достигнутым порогам уверенности.

4. Использование инструментов искусственного интеллекта

4.1 Место ИИ в архитектуре

ИИ-контур подключён к основному потоку асинхронно. После того как детерминированный путь обработки алерта завершён (сохранение, корреляция, решение), запускается отделённая задача обогащения: запрос публикуется в топик запросов, результат — в топик результатов. Разделение топиков сделано заранее, чтобы масштабируемая группа обработчиков могла заменить встроенный обработчик без изменения контракта.

Ключевое свойство контура — вызов модели не создаёт обратного давления на приём алертов. Медленная или недоступная модель задерживает только собственный результат.

4.2 Реализованные ИИ-сценарии

1. **Обогащение алерта.** Один вызов модели с ограниченным JSON-ответом возвращает классификацию, приоритет, гипотезу первопричины, уверенность и объяснение. Значения классификации и приоритета ограничены объявленным словарём: ответ вне словаря отбрасывается и сохраняется только в объяснении, а не передаётся дальше как доверенный.

2. **Формулировка уведомления.** Один вызов модели [11] формирует заголовок, приоритет и первый шаг проверки для сообщения дежурному (около 3,4 с на используемом GPU). Любой отказ — недоступность модели, таймаут, некорректный JSON, приоритет вне словаря — приводит к детерминированному построению текста форматированием полей алерта. В полезной нагрузке уведомления фиксируется, каким путём получен текст, поэтому вопрос «как часто модель реально участвует» отвечается по данным, а не по логам.

Обе функции реализованы через единственный сетевой стык с моделью: второго способа вызвать модель в коде нет. Это делает контроль периметра и таймаутов проверяемым.

4.3 Соответствие требованиям БТ7

Соответствие реализации требованиям к интеллектуальной обработке приведено в таблице 6. Здесь и далее состояние обозначено цветом: **реализовано**, **реализовано частично**, **целевая архитектура**, **неприменимо**, **первоочередной риск**.

Таблица 6. Соответствие требованиям БТ7

ID	Требование	Реализация и статус
ПТ7.1	Режимы off, shadow, suggest, assisted auto, full auto	Режим задаётся флагом развёртывания; по умолчанию off. Реализованы off, shadow и suggest; режимы автоматизации зарезервированы контрактом и на пилоте не включаются
ПТ7.2	Уровень автоматизации, порог уверенности, fallback	Уверенность возвращается по контракту и приводится к отрезку [0; 1] независимо от ответа модели; fallback задан на каждом пути отказа

Продолжение таблицы 6

ID	Требование	Реализация и статус
ПТ7.3	ИИ не блокирует приём и доставку критичных алертов	Обогащение запускается после завершения детерминированного пути и выполняется как отделённая задача
ФТ7.1	Дедупликация с сохранением истории	Техническая дедупликация выполняется детерминированным автоматом с публикацией решения; смысловая дедупликация — задел ИИ-контура, автообъединение на пилоте отключено
ФТ7.2	Карантин шумовых алертов, запрет автоподавления critical/high	Подавление выполняется только по правилу нестабильности сигнала; понижение критичности моделью не применяется автоматически
ФТ7.3	Приоритизация	Приоритет предлагается моделью в словаре p1–p4; при отказе — детерминированное отображение критичности в приоритет
ФТ7.4	Маршрутизация по каталогу, владельцам и дежурствам	Реализована детерминированно по реестру маршрутизации; при отсутствии владельца уведомление не отправляется и фиксируется как немаршрутизируемое
ФТ7.5	Декомпозиция составных инцидентов	На пилоте отключена в соответствии с требованием; предпосылка появилась — инцидент реализован как проекция решений ядра
ФТ7.6	Объяснимость решения	Результат содержит объяснение, уверенность и признаки; версия политики корреляции фиксируется идентификатором политики
ФТ7.7	Обратная связь пользователя	Частично: подтверждение обработки инцидента реализовано, автор берётся из сессии; переопределение решения ИИ с указанием причины — нет
НФТ7.1	Аудит решений ИИ	Решения пишутся в отдельную гипертаблицу с увеличенным сроком хранения (по умолчанию 400 дней против 90 у событий и алертов)
НФТ7.2	Fallback на детерминированные правила	Обеспечен архитектурно: ядро не зависит от модели
НФТ7.3	Задержка ИИ не задерживает critical	Обеспечено отделённым вызовом и ограничением числа одновременных обращений к модели
НФТ7.4	Запрет передачи секретов и ПДн, недоверенный текст алерта	Инференс размещён в контуре; в модель передаются только технические поля алерта; ответ модели принимается только в пределах объявленного словаря

4.4 ИИ в процессе разработки

Инструменты на основе больших языковых моделей применялись при разработке для генерации и рефакторинга кода, подготовки тестовых сценариев и документации. Принятые ограничения: архитектурные решения и границы контуров определялись разработчиками, весь сгенерированный код проходил ревью и покрывался тестами, а формулировки требований и матрица трассировки поддерживались вручную.

5. Инфраструктура решения

5.1 Топология развёртывания

Решение развёртывается как набор независимых Docker Compose [12] стеков: каждый со своим именем проекта, собственной сетью и томами. Стеки поднимаются независимо, за исключением случаев, когда один присоединяется к сети другого. Платформа присоединяется к сетям генератора событий, Prometheus, Zabbix и TimescaleDB; веб-интерфейс — к сети платформы.

Наружу публикуются только те порты, которые действительно нужны: база данных host-порта не имеет вовсе и доступна исключительно платформе по внутренней сети. Распределение портов пилотного стенда приведено в таблице 7.

Таблица 7. Порты пилотного стенда

Порт	Стек	Назначение
8080	Zabbix	веб-интерфейс мониторинга
9090 / 9093	Prometheus	мониторинг и Alertmanager
8090	Генератор событий	приём событий (Kafka не публикуется)
8100	Платформа	API
8091	Веб-интерфейс	дашборд
8110	Уведомления	диспетчер доставки
8120	Бот TrueConf	приём запросов на доставку
80 / 443 / 4307	TrueConf Server	мессенджер
8443 / 8543	TLS-фронт	HTTPS для TrueConf и веб-интерфейса
—	TimescaleDB	только внутренняя сеть

Отдельного пояснения требует размещение коннекторов. Целевая схема — экземпляр клиента-коннектора на каждом узле с системой мониторинга: он устанавливается в сетевом сегменте этого узла и инициирует исходящее соединение к платформе, поэтому системе мониторинга не требуется маршрут до ядра. На пилотном стенде все системы мониторинга находятся на одном узле, и адаптеры выполняются внутри процесса платформы; переход к распределённой схеме не затрагивает контракты сообщений и код адаптеров, а только их упаковку и размещение.

5.2 Потребление ресурсов

Приведённые ниже значения — не оценка, а замер работающего пилотного стенда: виртуальная машина 16 vCPU, 62 ГБ ОЗУ, 133 ГБ диска и GPU Tesla T4 (15 ГБ), 15 контейнеров, аптайм 5 суток. Замер снят в режиме ожидания (без нагрузочного прогона), поэтому показывает базовое, а не пиковое потребление (таблица 8).

Таблица 8. Замеренное потребление ресурсов пилотного стенда

Компонент	CPU	ОЗУ	Примечание
Платформа (монолит)	0,4 %	65 МБ	Основная логика: приём, корреляция, API
Веб-интерфейс	< 0,1 %	13 МБ	Статика и обратный прокси
TimescaleDB	0,8 %	384 МБ	Растёт с объёмом истории
Kafka	181 %	1,2 ГБ	В простое занимает около 1,8 ядра — см. пояснение ниже
Генератор событий	0,4 %	40 МБ	Только на стенде
Инференс vLLM	1,2 %	4,0 ГБ	Плюс 13,6 из 15,4 ГБ видеопамяти под весами модели
TrueConf Server	0,5 %	1,25 ГБ	Внешняя система, обычно уже есть у заказчика
Контроллер домена	0,2 %	255 МБ	Синтетический справочник стенда
Zabbix (СУБД, сервер, веб, агент)	1,1 %	945 МБ	Источник событий, обычно уже есть
Prometheus	< 0,1 %	44 МБ	Источник событий и метрик
Сервис CMDDB	0,1 %	33 МБ	Развёрнут на стенде, платформой пока не используется
TLS-фронт	< 0,1 %	16 МБ	Терминация TLS
Итого	≈ 1,9 ядра	≈ 9 ГБ	Диск: 38 ГБ образов, 2,2 ГБ томов, всего занято 74 ГБ

Из замера следуют три вывода, влияющие на выбор конфигурации.

Во-первых, **прикладные компоненты почти ничего не потребляют**: платформа, интерфейс и диспетчер доставки вместе занимают менее 150 МБ ОЗУ и доли процента процессора. Ограничением является не прикладной код, а инфраструктура под ним.

Во-вторых, **основной потребитель процессора — шина сообщений**: Kafka в простое занимает около 1,8 ядра, то есть больше, чем все остальные контейнеры вместе. Это конфигурация стенда с одним брокером и настройками по умолчанию; для промышленной эксплуатации требуется отдельный узел и настройка, а не соседство с прикладными сервисами.

В-третьих, **III-контур ограничен видеопамятью, а не процессором**: модель занимает 13,6 из 15,4 ГБ видеопамяти постоянно, независимо от нагрузки, при загрузке GPU 0 % в простое. Это и есть причина размещать инференс отдельно: он масштабируется другой величиной (видеопамять и число одновременных запросов), чем всё остальное.

5.3 Целевая конфигурация

Схема развёртывания приведена на рисунке 10: виртуальные машины, контейнеры внутри них и направление роста каждого узла.

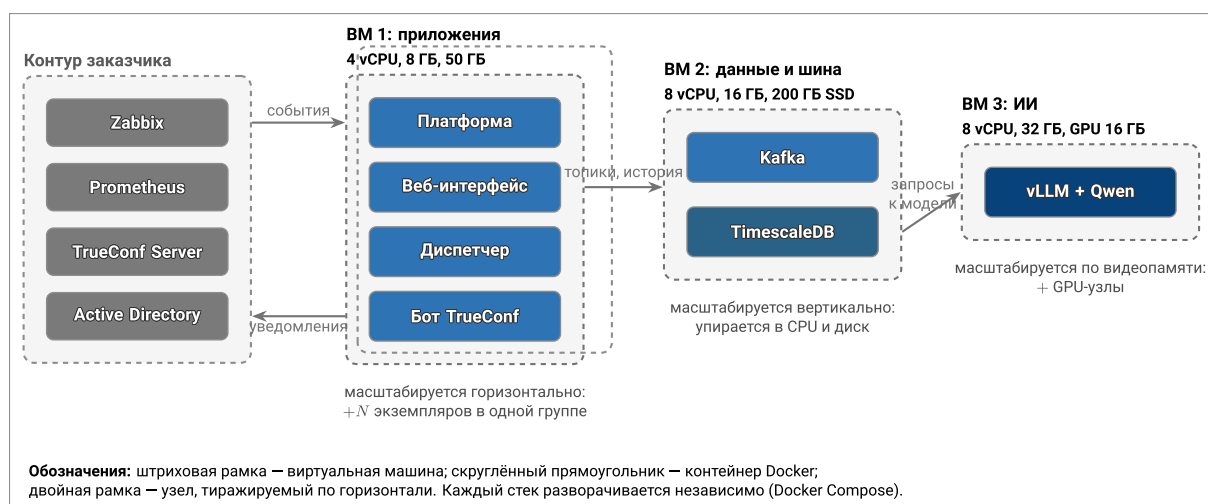


Рисунок 10. Развёртывание и масштабирование: узлы, контейнеры, направление роста

Развёртывание планируется раздельным: монолиты платформы отдельно, ИИ-сервисы отдельно, данные и шина отдельно. Такое разделение отражает разные профили потребления из предыдущего раздела и разные требования к масштабированию. Целевые конфигурации приведены в таблице 9.

Таблица 9. Целевая конфигурация узлов

Узел	Состав	Конфигурация	Чем ограничен
Приложения	Платформа, веб-интерфейс, диспетчер доставки, бот, коннекторы	4 vCPU, 8 ГБ ОЗУ, 50 ГБ	Числом экземпляров, не ресурсами; запас ≈ 50 -кратный к замеру
Данные и шина	Kafka, TimescaleDB	8 vCPU, 16 ГБ ОЗУ, 200 ГБ SSD	Процессором (шина) и диском (история); растёт с retention
ИИ	vLLM, будущие обработчики	8 vCPU, 32 ГБ ОЗУ, GPU 16 ГБ, 100 ГБ	Видеопамятью и числом одновременных запросов
Интеграции	Zabbix, Prometheus, каталог, TrueConf	Ресурсы заказчика	Обычно уже развёрнуто; в расчёт затрат не включается

Разделение даёт три эксплуатационных следствия: прикладной узел можно перезапускать и масштабировать, не трогая шину и историю; отключение или перезагрузка ИИ-узла не влияет на приём и доставку (это же свойство проверено тестами); а GPU — самый дорогой ресурс — выделяется только под инференс и не простаивает в составе прикладного узла.

5.4 Развёртывание

Конфигурация разделена на два уровня: версионизируемый файл флагов (версии образов, порты, тайм-ауты, режимы) и приватный файл секретов, который генерируется на целевом узле и никогда не попадает в репозиторий и не перезаписывается при развёртывании. Развёртывание выполняется скриптом: синхронизация стека, запуск, проверка состояния контейнеров и обращение к адресу проверки работоспособности с однозначным результатом.

5.5 Модель масштабирования

Масштабирование выполняется добавлением экземпляров платформы и обработчиков: это участники одной группы потребителей Kafka, разделяющие партиции. Kafka при этом не масштабируется — она является буфером, а не узким местом; узкими местами являются СУБД и инференс модели, и их масштабирование — отдельное решение.

Принятый компромисс формулируется явно: это масштабирование виртуальными машинами, а не оркестратором. Узел является одновременно единицей масштаба и единицей отказа: при потере узла его контейнеры остаются недоступны до ручного переразвёртывания. Такое решение даёт рост пропускной способности, но не отказоустойчивость; при появлении требования к непрерывности потребуется оркестратор. До запуска второго экземпляра платформы требуется вынести во внешнее хранилище состояние ограничителя запросов и кэш живого хвоста.

5.6 Наблюдаемость

Состав наблюдаемости и подход к дежурству опираются на практики эксплуатации крупных production-систем [13, 14]. Каждый стек публикует метрики в формате Prometheus и предоставляет адрес проверки работоспособности, включающий состояние подключения к Kafka и к базе. Метрики уведомлений размечены исходом и способом формирования текста, что позволяет отслеживать долю немаршрутизируемых алертов и фактический вклад модели.

6. Информационная безопасность

6.1 Подход и границы моделирования

Модель угроз описывает, *кто* и *каким способом* может нарушить работу платформы, и служит основанием для выбора защитных мер. Она отвечает на вопрос «что может пойти не так», тогда как оценка ущерба и приоритет устранения — предмет реестра рисков и вынесены в раздел 6.13. Угроза является входом для риска: перечень угроз первичен, приоритизация — вторична.

Границы моделирования: рассматривается платформа управления оповещениями и её интерфейсы с внешними системами. Сами системы мониторинга, мессенджер и каталог считаются внешними и защищаются собственными средствами; в модели они присутствуют как источники недоверенных данных и как объекты, доступ к которым может быть использован против платформы.

Объектами защиты являются: достоверность и целостность событий; доступность критического пути доставки; целостность решений о критичности и адресате; разграничение доступа к данным мониторинга; доказуемость действий (аудит); конфиденциальность секретов и технических идентификаторов.

Классификация угроз для интеллектуальных компонентов опирается на OWASP Top 10 для приложений с большими языковыми моделями [15] и рамочную модель управления рисками ИИ [16]; требования к веб-контуре — на OWASP ASVS [17]. Для промышленной эксплуатации на объектах критической информационной инфраструктуры дополнительно применяются 187-ФЗ [18] и приказ ФСТЭК России № 239 [19], а система менеджмента ИБ — ГОСТ Р ИСО/МЭК 27001-2021 [20].

6.2 Модель нарушителя

Категории нарушителей и их возможности приведены в таблице 10. Категории упорядочены по возрастанию возможностей; нарушитель более высокой категории обладает всеми возможностями предыдущих.

Таблица 10. Категории нарушителей

Код	Нарушитель	Возможности	Мотивация
H1	Внешний, без доступа к корпоративной сети	Обращение к опубликованным точкам входа: веб-интерфейс, TLS-фронт	Отказ в обслуживании, разведка инфраструктуры
H2	Внешний, с доступом к сегменту мониторинга	Отправка произвольных сообщений на точки приёма webhook, наблюдение внутреннего трафика сегмента	Подделка или сокрытие аварий
H3	Внутренний пользователь платформы (наблюдатель, инженер)	Аутентифицированный доступ к данным мониторинга в пределах своей роли	Превышение прав, доступ к чужим объектам
H4	Внутренний администратор платформы	Изменение ролей, правил маршрутизации, режимов ИИ и подписок	Соккрытие собственных действий, злоупотребление правами

Продолжение таблицы 10

Код	Нарушитель	Возможности	Мотивация
Н5	Администратор инфраструктуры (узел, контейнеры)	Доступ к файлам конфигурации, секретам, томам и журналам	Доступ к данным в обход прикладных проверок
Н6	Поставщик программных компонентов	Внесение изменений в зависимости, образы и веса модели до их попадания в контур	Закрепление в контуре, скрытая функциональность
Н7	Автор текста алерта (косвенный нарушитель)	Формирование содержимого события, попадающего в модель как данные	Манипуляция решением интеллектуального контура

Нарушитель Н7 выделен отдельно, поскольку он не имеет никакого доступа к системе: его единственный инструмент — содержимое поля алерта, которое платформа обязана принять. Именно эта категория делает обязательным правило «текст алерта — недоверенные данные».

6.3 Перечень угроз

Угрозы приведены в таблице 11 в форме «угроза — вектор реализации — меры противодействия». Столбец мер перечисляет полный требуемый набор, а столбец состояния показывает, какая его часть реализована: нейтрализована, нейтрализована частично, не нейтрализована либо неактуальна для текущей архитектуры с указанием причины. Развёрнутое описание мер по контурам приведено в разделах 6.5–6.12.

Таблица 11. Перечень угроз безопасности информации

Код	Наруш.	Вектор реализации	Последствие	Меры противодействия	Состояние
У-01	H2	Отправка сфабрикованных событий на неаутентифицированную точку приёма webhook	Ложные аварии, отвлечение дежурных, маскировка реальной аварии	Отдельная идентичность источника, взаимная аутентификация или подпись запроса, allowlist адресов, аудит подключений	Не нейтрализована: аутентификация источника не реализована
У-02	H2	Массовая отправка событий (лавина) с целью исчерпания обработки	Задержка или потеря критичных уведомлений	Ограничение частоты на входе, буферизация в шине, подавление нестабильных ключей, приоритетные очереди для critical, квоты по источникам	Частично: ограничение частоты и буферизация есть, приоритетных очередей нет
У-03	H2	Повторная отправка ранее принятого события	Дублирование инцидентов, повторные уведомления	Детерминированные идентификаторы сообщения и корреляции, дедупликация в автомате, проверка идемпотентности на приёме	Частично: идентификаторы детерминированы, отдельной проверки идемпотентности приёма нет
У-04	H2	Передача некорректной или вредоносной структуры данных вендора	Отказ обработчика, порча внутреннего состояния	Строгая схема входных данных, отказ 422 вместо интерпретации, изоляция адаптеров, запрет двух адаптеров на один идентификатор	Нейтрализована
У-05	H1	Перебор и автоматизированные обращения к API и точкам аутентификации	Отказ в обслуживании, подбор учётных данных	Раздельные политики ограничения частоты для webhook и API, TLS на периметре, WAF и защита от перебора	Частично: ограничение частоты по префиксам есть, WAF отсутствует
У-06	H7	Внедрение инструкций в текст алерта, попадающий в модель	Изменение классификации, приоритета или адресата	Разделение инструкции и данных в запросе, приём ответа только в пределах словаря, отсутствие полномочий у модели, критичные решения вне модели	Нейтрализована
У-07	H7	Формирование текста, вызывающего аномально долгую обработку моделью	Задержка обогащения, исчерпание слотов GPU	Отделённый вызов, тайм-аут, ограничение числа параллельных обращений, разрешение получателя до обращения к модели	Нейтрализована
У-08	—	Недоступность или деградация модели	Остановка обработки при синхронной зависимости	Детерминированное ядро, fire-and-forget вызов, fallback на форматированные поля, флаг отключения контура	Нейтрализована архитектурно, подтверждено тестами

Продолжение таблицы 11

Код	Наруш.	Вектор реализации	Последствие	Меры противодействия	Состояние
У-09	Н3	Обращение к данным и объектам за пределами своей роли	Раскрытие сведений об инфраструктуре и авариях	Аутентификация для всех разделов мониторинга, ролевая модель, объектная авторизация по владельцу и команде	Частично: ролевая модель есть, объектной авторизации нет
У-10	Н1, Н3	Кража сессионного токена (перехват, межсайтовый сценарий)	Действия от имени пользователя	TLS, httponly-cookie, передача токена во фрагменте URL, строгий список адресов возврата, второй фактор, ограничение времени жизни сессии	Частично: второго фактора нет
У-11	Н4	Изменение правил маршрутизации или подписок, чтобы не уведомлять о конкретной аварии	Соккрытие инцидента	Журналирование всех изменений правил и подписок, обязательные подписки на critical, разделение полномочий, оповещение о смене режима ИИ	Частично: журналирование есть, неизменяемого хранилища аудита нет
У-12	Н4, Н5	Изменение или удаление записей аудита	Невозможность разбора инцидента, отрицание действий	Отдельное хранилище аудита в режиме только на дозапись, ограниченный круг пишущих компонентов, увеличенный срок хранения	Частично: увеличенный срок хранения решений, режим только на дозапись не реализован
У-13	Н5	Извлечение секретов из файлов конфигурации, переменных окружения или журналов	Доступ к базе, мессенджеру, API мониторинга	Централизованное хранилище секретов, внедрение в среду выполнения, ротация, маскирование в журналах, автоматическая проверка на секреты	Частично: секреты вне репозитория, хранилища и ротации нет
У-14	Н5	Горизонтальное перемещение между контейнерами и сетями стенда	Расширение зоны поражения	Сегментация сетей, запуск не от суперпользователя, запрет привилегированного режима, ограничение возможностей ядра, файловая система только для чтения	Частично: сегментация есть, ограничения привилегий не применены
У-15	Н5	Прямое обращение к хранилищу в обход прикладных проверок	Чтение и изменение истории и ролей	Отсутствие публикуемого порта у СУБД, отдельные сервисные учётные записи с минимальными правами, шифрование и резервное копирование	Частично: порт не публикуется, отдельных учётных записей нет
У-16	Н6	Подмена образа, зависимости или весов модели	Скрытая функциональность в контуре	Фиксация версий, статический анализ и анализ зависимостей, сканирование образов, состав поставки, подпись образов, реестр моделей с проверкой контрольной суммы	Не нейтрализована: внедрена только фиксация версий

Продолжение таблицы 11

Код	Наруш.	Вектор реализации	Последствие	Меры противодействия	Состояние
У-17	Н1, Н6	Передача данных во внешние сервисы моделей	Нарушение требования о размещении в контуре	Инференс внутри контура, запрет исходящего доступа ИИ-зоны в интернет, инвентаризация моделей	Нейтрализована
У-18	—	Отказ узла, на котором размещены контейнеры	Простой платформы и прекращение доставки	Несколько экземпляров критичных сервисов, оркестратор, аварийный обход на прежний механизм оповещения, резервное копирование и тесты восстановления	Не нейтрализована: один узел, обход не реализован
У-19	—	Молчание источника событий (отказ или отключение)	Незамеченная потеря наблюдаемости	Контроль отсутствия ожидаемого потока по каждому источнику, оповещение при превышении интервала тишины	Не нейтрализована
У-20	—	Отказ записи в хранилище истории	Потеря аудиторского следа при сохранении обработки	Изоляция сбоя от потока обработки, метрика состояния хранилища, подтверждение приёма после надёжного сохранения	Частично: сбой изолирован и логируется
У-21	Н3, Н5	Получение персональных данных из базы платформы	Нарушение требований к обработке ПДн	Хранение только технических псевдонимизированных идентификаторов, синтетические учётные записи, сроки хранения	Неактуальна: ПДн не обрабатываются
У-22	Н3	Доступ к чужому контексту через поиск по базе знаний	Раскрытие сведений смежных подразделений	Проверка прав до поиска, разделение источников по уровням доверия, очистка перед векторизацией	Неактуальна: поиск по базе знаний не используется
У-23	Н2, Н4	Компрометация учётных данных интеграции с каталогом или CMDB	Доступ к корпоративным справочникам	Отдельная сервисная учётная запись только на чтение, allowlist атрибутов, TLS, ротация, аудит обращений	Неактуальна на пилоте: обращений к CMDB и каталогу нет

Из 23 угроз шесть нейтрализованы полностью, одиннадцать — частично, три не нейтрализованы и требуют реализации мер до промышленной эксплуатации, три неактуальны для текущей архитектуры. Полностью нейтрализованы именно угрозы, связанные с интеллектуальным контуром (У-06 – У-08, У-17): это результат архитектурного решения не давать модели полномочий, а не следствие дополнительных защитных средств.

6.4 Границы доверия

Контрольные границы и минимальный набор контролей на каждой из них приведены в таблице 12; столбец состояния показывает, что из этого работает на пилоте.

Таблица 12. Границы доверия и контроли

Граница	Назначение	Минимальные контроли	Состояние
ТВ1: мониторинг → платформа	Приём событий от внешних источников	Аутентификация источника, валидация схемы, TLS, ограничение частоты, аудит	Частично: схема, ограничение частоты и TLS есть; аутентификации источника нет
ТВ2: пользователь → платформа	Пользовательский и административный доступ	Единый вход, авторизация по ролям, защита от подделки запросов и внедрения разметки, аудит	Частично: вход и роли есть; второй фактор и объектная авторизация — целевые
ТВ3: платформа → каталог и CMDB	Доступ к корпоративным справочникам	Учётные записи только на чтение, список разрешённых атрибутов, TLS, аудит	Целевое: сервис CMDB развёрнут на стенде, но платформой не используется; AD — номинальный справочник
ТВ4: платформа → TrueConf	Исходящие уведомления и подтверждения	Отдельная сервисная идентичность, ротация секретов, подписанные подтверждения, аудит	Частично: отдельная учётная запись бота есть; подтверждений нет
ТВ5: ядро → ИИ	Передача данных в интеллектуальный контур	Недоверенный вход, структурированный выход, ограничение полномочий, резервный путь	Реализовано
ТВ6: приложение → данные	Доступ приложений к данным	Минимальные привилегии, сегментация, шифрование, резервное копирование, аудит	Частично: сегментация и закрытая БД есть; отдельных ролей БД и резервного копирования нет

6.5 Недоверенный входной контур

Любое событие от внешнего источника рассматривается как недоверенные данные. Доверие формируется не содержанием события, а подтверждённой идентичностью источника и прохождением контролей входного контура.

Реализовано на пилоте: строгая схема входных данных (некорректная нагрузка отклоняется кодом 422, а не интерпретируется), отдельное ограничение частоты для машинных webhook и пользовательского API, изоляция адаптеров друг от друга (каждый владеет только своим форматом), детерминированные идентификаторы сообщения и корреляции,

выводимые из полей вендора — это обеспечивает устойчивость к повторной доставке на уровне корреляции.

Не реализовано и вынесено в план: аутентификация источника (сейчас точка приёма webhook доступна без учётных данных, что допустимо на изолированном стенде и недопустимо в промышленной эксплуатации), управляемый жизненный цикл подключения источника, отдельная очередь неразбираемых сообщений.

6.6 Безопасный контур ИИ

ИИ рассматривается как вспомогательный компонент принятия решений, а не как самостоятельная точка доверия. Это требование выполнено конструктивно и является сильной стороной архитектуры:

- текст алерта отделён от системной инструкции и передаётся как данные;
- выход модели принимается только в пределах объявленного словаря; значение вне словаря отбрасывается, а не передаётся дальше как доверенное;
- уверенность приводится к отрезку $[0; 1]$ независимо от того, что вернула модель;
- модель не имеет полномочий на действия: она не изменяет каталог, базу и подписки и не отправляет уведомления — её результат является рекомендацией, которую применяет детерминированный код;
- критичные решения принимаются правилами вне модели; автоматическое понижение или скрывание критичных алертов не предусмотрено;
- при недоступности или ошибке модели применяется детерминированный резервный путь, а сам контур отключается флагом без остановки обработки.

Отдельного компонента-политики, ограничивающего действия модели, в системе нет и не требуется: у модели нет ни одного канала воздействия, который такой компонент мог бы ограничивать. Если в будущем модели будут переданы полномочия на действия, ограничитель политик становится обязательным.

Домен «базы знаний и поиск контекста» к текущей архитектуре *неприменим*: поиск по корпоративным документам и векторные представления не используются, в модель передаются только технические поля одного алерта. Требования этого домена (проверка прав до поиска, разделение источников по уровням доверия, очистка перед векторизацией) становятся обязательными при добавлении поиска по базе знаний и зафиксированы как условие такого расширения.

6.7 Интеграции и административный контур

Каждая интеграция имеет отдельную сервисную идентичность и минимальный набор разрешений: для API мониторинга используется выделенный токен сервисной учётной записи с доступом только на чтение, для мессенджера — отдельная учётная запись бота. Пароль администратора в качестве прикладной учётной записи не используется.

Административные операции доступны только роли администратора, и проверка полномочий выполняется на стороне сервера, а не ограничением интерфейса. Разделы с дан-

ными мониторинга закрыты аутентификацией: без сессии интерфейс показывает только экран входа и системное состояние (рисунок 11).

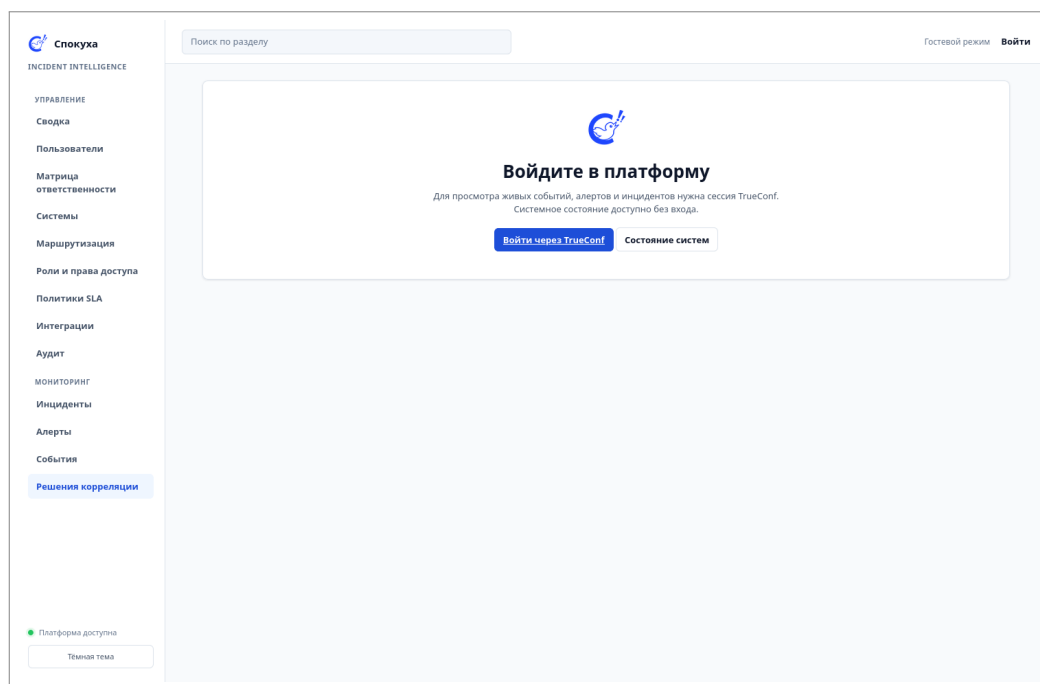


Рисунок 11. Закрытие разделов мониторинга: вход через TrueConf

Межоригинный вход выполняется по строгому списку разрешённых адресов возврата, причём токен передаётся во фрагменте URL, который принципиально не передаётся на сервер.

Объектная авторизация (доступ к конкретному инциденту, активу, подписке) и второй фактор аутентификации отнесены к целевой архитектуре: объектной модели инцидентов и подписок на пилоте нет, а второй фактор определяется политикой корпоративного провайдера входа.

6.8 Доступность критического пути

Ключевое требование — сохранение доставки критичного события при отказе отдельных компонентов. Архитектурно критический путь не зависит от языковой модели: вызов модели отделён и не создаёт обратного давления, а отказ записи в хранилище не выходит в поток обработки. Шина сообщений отделяет приём от обработки и позволяет пережить всплеск без потери событий.

Не реализовано: приоритетные очереди для критичных алертов (сейчас общий поток), несколько экземпляров критичных сервисов, отдельная очередь неразбираемых сообщений, технический аварийный обход на существующий механизм оповещения. На пилоте роль обхода выполняет организационная мера — дублирование критичных уведомлений прежним способом.

6.9 Данные, секреты и журналы

Таблица 13. Контроли данных, секретов и журналов

Область	Требуемый контроль	Состояние
Секреты	Централизованное хранилище, внедрение в среду выполнения, ротация, запрет хранения в репозитории	Частично: секреты вне репозитория, генерируются случайно, не перезаписываются при развёртывании; централизованного хранилища и ротации нет
Операционные данные	Раздельные сервисные учётные записи БД с минимальными правами	Не реализовано: одна учётная запись приложения
Аудит	Отдельное хранилище только на дозапись с ограниченным кругом пишущих компонентов	Частично: журнал решений хранится отдельной таблицей с увеличенным сроком (400 дней), но не изолирован как хранилище
Логи	Маскирование токенов, паролей и чувствительных ответов; структурированный формат	Частично: структурированные логи есть, автоматической проверки на секреты нет
Персональные данные	Хранение только технических идентификаторов	Реализовано: ФИО, телефоны и адреса не хранятся
Сроки хранения	Отдельные сроки для событий, алертов, доставки, аудита и решений	Реализовано для событий, алертов и решений

6.10 Сетевая и контейнерная изоляция

Сетевая модель ограничивает возможность перемещения между компонентами: интеграции, приложения, данные и ИИ разнесены по разным сетям, хранилище и шина не публикуют портов наружу, ИИ-контур не имеет исходящего доступа к внешним сервисам моделей. Представление зон приведено на рисунке 12.

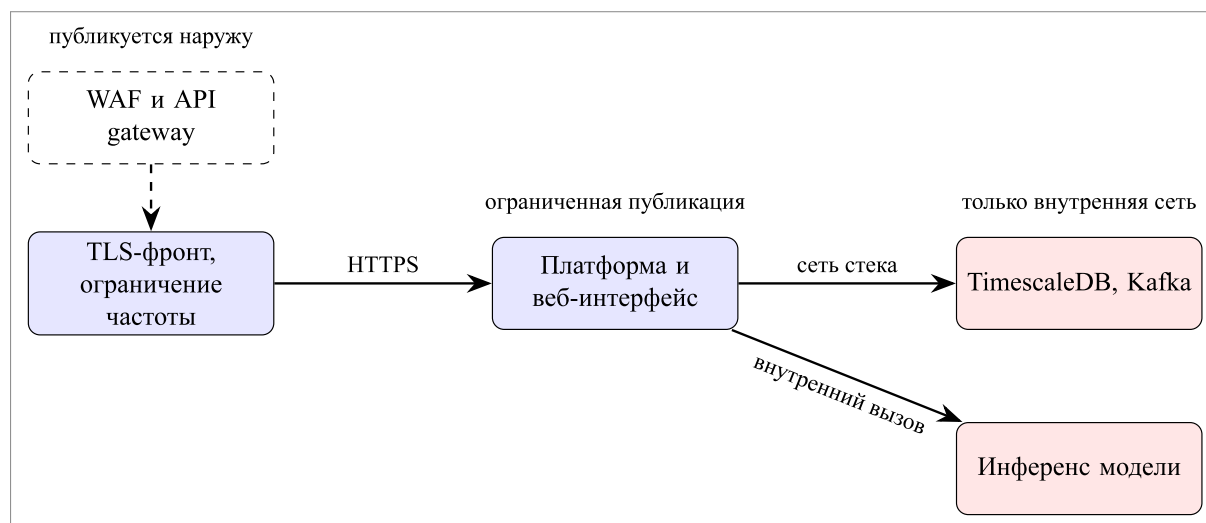


Рисунок 12. Сетевые зоны и публикуемые точки входа (штриховая рамка — целевая архитектура)

Контейнерная защита реализована частично. Каждый стек имеет собственную сеть и тома, что ограничивает зону поражения. Требования принципа наименьших привилегий — запуск не от суперпользователя, запрет привилегированного режима, ограничение возможностей ядра, файловая система только для чтения — на пилоте не применены; более того, два вспомогательных стека (агент мониторинга и контроллер домена) работают в

привилегированном режиме, что приемлемо для стенда и требует пересмотра до промышленной эксплуатации.

6.11 Цепочка поставки

Контроли доверенности артефактов (статический анализ, анализ зависимостей, сканирование образов и секретов, состав поставки, подпись образов, реестр моделей с проверкой контрольной суммы) на пилоте не внедрены. Зафиксированы версии образов в версиионируемом файле флагов, что обеспечивает воспроизводимость, но не доверенность. Домен отнесён к очереди подготовки к промышленной эксплуатации.

6.12 Мониторинг безопасности

Каждый компонент публикует метрики и адрес проверки работоспособности, включающий состояние подключения к шине и базе. Метрики уведомлений размечены исходом, что позволяет отслеживать долю немаршрутизируемых алертов.

Особое значение имеет контроль отсутствия ожидаемого потока: молчание источника неотличимо от исправной работы, если его не контролировать отдельно. Такой контроль, как и обнаружение всплесков отказов авторизации и массовых изменений маршрутизации и подписок, отнесён к целевой архитектуре.

6.13 Нейтрализованные угрозы и очередность работ

Здесь угрозы из таблицы 11 переводятся в план работ: для каждой неделанной меры указано фактическое состояние и очередь устранения. Сводка приведена в таблице 14. Столбец очереди соответствует приоритизации реестра: P0 — закрывается до промышленной эксплуатации, P1 — до вывода в производственный контур.

Таблица 14. Нейтрализованные угрозы: состояние и очередность

Требование реестра	Фактическое состояние	Оч.	Влияние
Аутентификация каждого источника событий	Точка приёма webhook доступна без учётных данных; защищена только ограничением частоты и изоляцией сети стенда	P0	Подделка событий, ложные аварии
Подтверждение приёма только после надёжного сохранения	Событие публикуется в шину до ответа источнику; отказ записи в проекцию логируется и не поднимается	P0	Возможна потеря истории при отказе хранилища
Приоритетные очереди для критичных алертов	Общий поток без приоритизации	P0	Задержка критичного при лавине
Отдельная очередь неразбираемых сообщений	Не реализована	P0	Повторная обработка некорректных сообщений
Объектная авторизация и второй фактор	Роли есть, объектной модели и второго фактора нет	P0	Избыточный доступ администратора
Подписанное подтверждение обработки	Подтверждение реализовано, автор берётся из сессии, а не из тела запроса; криптографической подписи нет	P1	Ограниченная доказуемость реакции
Централизованное хранилище секретов и ротация	Секреты в частных файлах вне репозитория, ротации нет	P0	Компрометация при доступе к узлу

Продолжение таблицы 14

Требование реестра	Фактическое состояние	Оч.	Влияние
Технический аварийный обход на прежний механизм	Отсутствует; на пилоте применяется организационное дублирование	P0	Простой оповещения при отказе платформы
Контроль отсутствия потока от источника	Не реализован	P0	Незамеченное молчание источника
Раздельные учётные записи БД, отдельное хранилище аудита	Одна учётная запись приложения; журнал решений в общей БД	P1	Смешение операционных и аудиторских данных
Контейнерная защита: без суперпользователя, без привилегий, ограничение возможностей	Не применена; два вспомогательных стека привилегированные	P1	Расширение зоны поражения
Резервное копирование и регулярные тесты восстановления	Не настроены	P1	Невосстановимость данных
Несколько экземпляров критичных сервисов	Один узел, единая точка отказа	P1	Простой при отказе узла
Контроли цепочки поставки	Не внедрены; зафиксированы версии образов	P1	Уязвимые или подменённые артефакты
Ограничение прав интеграций с CMDB и каталогом	Платформа не обращается к CMDB и каталогу; требование станет применимым при подключении	—	Неприменимо на пилоте
Контроли поиска по базам знаний	Поиск по базе знаний и векторные представления не используются	—	Неприменимо к текущей архитектуре

Отдельно отмечается, что часть требований реестра выполнена конструктивно и не требует дополнительных мер: отсутствие персональных данных, отсутствие полномочий у модели, независимость критичного пути от модели, отсутствие исходящего доступа ИИ-контура к внешним сервисам.

6.14 Доказательство эффективности защитных мер

Для контролей первой очереди декларативного описания недостаточно: защита подтверждается функциональным тестом или отрицательным сценарием. Текущее состояние доказательной базы приведено в таблице 15.

Таблица 15. Проверяемые утверждения о защите

Утверждение	Способ подтверждения
Отключение ИИ не останавливает обработку и доставку критичного алерта	Автоматический тест: подтверждено
Ответ модели вне объявленного словаря не применяется	Автоматический тест: подтверждено
Отказ модели, тайм-аут и некорректный JSON приводят к детерминированному тексту	Автоматический тест: подтверждено
Некорректная нагрузка webhook отклоняется, а не интерпретируется	Автоматический тест: подтверждено
Два адаптера не могут занять один идентификатор источника	Автоматический тест: подтверждено

Продолжение таблицы 15

Утверждение	Способ подтверждения
Разделы мониторинга недоступны без аутентификации	Автоматический тест: подтверждено
Отказ записи в хранилище не прерывает обработку	Автоматический тест: подтверждено
Неаутентифицированный источник получает отказ	Требуется реализации аутентификации источника
Повторная доставка одного события не создаёт новый инцидент	Частично: идентификаторы детерминированы, отдельного теста идемпотентности приёма нет
Критичная очередь сохраняет приоритет при лавине	Требуется реализации приоритетных очередей
Секреты отсутствуют в конфигурации и журналах	Требуется автоматической проверки
Отказ платформы допускает управляемый переход на прежний механизм	Требуется реализации обхода

7. Экономика

7.1 Методика и вводные данные

Финансово-экономическая модель построена по схеме дерева КПЭ: продуктовые метрики — факторы влияния (уровень 4) — физические параметры (уровень 3) — операционные КПЭ (уровень 2) — инвестиционные КПЭ (уровень 1). Принципиальное отличие настоящего расчёта от оценочного состоит в том, что нижний уровень дерева не постулируется, а измеряется: значения факторов влияния сняты прогоном эталонного корпуса через реальный конвейер обработки, а инварианты, без которых эффект недостижим, зафиксированы автоматическими тестами.

Все числа модели рассчитываются скриптом `context/economics.py`: допущения заданы в нём константами, а таблицы и выводы главы приводят его результат. Это исключает арифметические расхождения между разделами и позволяет пересчитать модель под другие ставки одной командой.

Вводные данные, обязательные к использованию, приведены в таблице 16; собственные допущения выделены отдельно (таблица 18) и снабжены обоснованием.

Таблица 16. Обязательные вводные данные

Параметр	Значение	Применение
Стоимость ФТЕ (с учётом взносов и накладных расходов)	2 000 руб./ч	Расчёт эффектов
Стоимость часа работы проектной команды	3 500 руб./ч	Расчёт расходной части
Стоимость простоя установки НПЗ	150 млн руб./сут. (6,25 млн руб./ч)	Расчёт эффекта от сокращения простоя

7.2 Источники измерения

Симуляция. Эталонный корпус (1053 алерта, 8 инцидентов, 2 часа модельного времени, фиксированный seed) прогоняется через реальный конвейер. Эталонная разметка в конвейер не попадает: в метки алерта передаются только сервис, среда, площадка, объект и команда-владелец, а признаки «значимый/шумовой» и идентификатор инцидента доступны только измерителю. Иначе корреляция сгруппировала бы алерты по идентификатору инцидента и показала бы идеальный результат по неверной причине.

Тесты. Автоматические тесты фиксируют инварианты, на которых держится модель эффекта: недоступность языковой модели не прерывает доставку, отказ записи в хранилище не выходит в поток обработки, уверенность всегда лежит в отрезке $[0; 1]$, ответ модели вне словаря отбрасывается. Текущее состояние — 145 пройденных тестов платформы, 1 пропущенный (требует развёрнутой модели), 4 отмеченных как известные ограничения; отдельно покрыты диспетчер доставки и генератор событий. Расчёт эффекта предполагает, что критичное уведомление доходит и при отказе ИИ — это утверждение проверяется тестом, а не декларируется.

7.3 Продуктовые метрики (база и цель)

Продуктовые метрики и факторы влияния уровня 4 — это одни и те же величины, снятые с симуляции (таблица 17). Целевые значения задаются относительно измеренной базовой линии и снимаются тем же способом: на корпусе — прогоном измерителя, в эксплуатации — метриками платформы.

Таблица 17. Продуктовые метрики: базовое и целевое значения

Метрика	База	Достигнуто	Цель	Способ измерения
Уведомлений за 2 ч корпуса, шт.	1053	44	≤ 50	Измеритель на корпусе
Уведомлений в час, шт.	527	22	≤ 25	Метрики платформы
Уведомлений на один инцидент, шт.	132	5,5	≤ 6	Измеритель; метрики решений
Шумовых уведомлений, шт.	508	17	≤ 10	Измеритель по эталону
Уведомлений не тому адресату, шт.	329	13	≤ 5	Измеритель; обратная связь дежурных
Доля полезных уведомлений	20,5 %	31,8 %	$\geq 60 \%$	Измеритель по владельцу сервиса
Полнота обнаружения инцидентов	8/8	8/8	8/8	Измеритель; сверка с журналом аварий
Время до первого уведомления	—	—	≤ 15 с	Метрики платформы
Доля немаршрутизируемых алертов	—	—	$\leq 5 \%$	Метрика маршрутизации
Доставка при недоступности ИИ	—	100 %	100 %	Тесты отказа модели

Полнота вынесена в таблицу как контрольная метрика: снижение числа уведомлений при потере хотя бы одного инцидента из восьми было бы неприемлемо для промышленной эксплуатации. Именно поэтому правило эскалации, восстановившее полноту до 8/8 ценой восьми дополнительных уведомлений и 0,8 процентного пункта снижения шума, является правильным разменом, а не ухудшением результата. Достижение целевой доли полезных уведомлений (60 %) требует учёта первопричины при маршрутизации: измерение показало, что 13 из 44 уведомлений уходят дежурному сигнализирующего, а не сломавшегося сервиса. Экономическая цель и техническая задача следующего этапа здесь — одно и то же утверждение, проверяемое на том же корпусе.

7.4 Собственные допущения

Таблица 18. Собственные допущения и их обоснование

Параметр	Значение	Обоснование
Поток алертов пилотного периметра	12 636 шт./сут.	Экстраполяция измеренного корпуса на сутки ($\times 12$)
Среднее время реакции на уведомление	2 мин	Оценка времени открыть, прочитать и квалифицировать оповещение
Дежурные смены	2 по 8 ч	Типовая организация дежурства на пилотном периметре
Доля времени дежурного на разбор оповещений	30 %	Ограничивает эффект сверху фактическим фондом времени
Доля высвобождаемого времени	70 %	Часть разбора остаётся при любом снижении шума
Значимых инцидентов с простоем в год	6	Пилотный периметр; уточняется по журналу аварий
Сокращение времени обнаружения и адресации на инцидент	15 мин	Следствие снижения со 132 до 5,5 уведомлений на инцидент и адресной доставки
Состав проектной команды	6 человек	Архитектор, три разработчика, специалист по ИБ, аналитик
Трудоёмкость разработки и внедрения пилота	1 920 ч	6 человек $\times$ 8 недель $\times$ 40 ч
Трудоёмкость сопровождения	30 ч/мес	Поддержка адаптеров, правил и конфигурации
Инфраструктура: три узла (приложения, данные и шина, ИИ с GPU)	0,76 млн руб./год	Конфигурации из таблицы 9; расчёт стоимости — таблица 20
Ставка дисконтирования	20 %	Типовая для ИТ-проектов внутренней автоматизации
Горизонт расчёта	3 года	Соответствует сроку жизни пилотного решения до пересмотра

7.5 Дерево КПЭ

Переход от измеренных метрик к деньгам выполняется в два шага: факторы влияния пересчитываются в физические параметры (человеко-часы, часы простоя), а те — в операционные КПЭ по обязательным ставкам (рисунок 13).

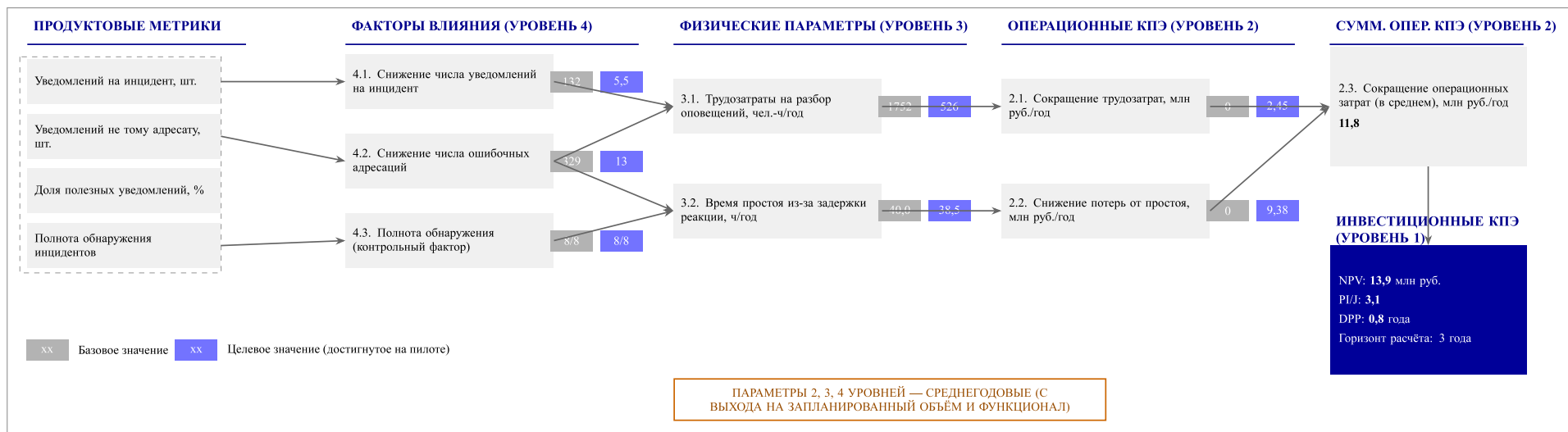


Рисунок 13. Дерево КПЭ: от продуктовых метрик к инвестиционным показателям

Физический параметр 3.1 — сокращение трудозатрат на разбор оповещений. Прямое умножение сокращённого потока (12 204 уведомления в сутки) на время реакции даёт 407 человеко-часов в сутки, что заведомо превышает фонд рабочего времени дежурных. Такой расчёт был бы неверен: эти часы сегодня не тратятся, поток игнорируется — и именно через это теряются аварии. Поэтому эффект ограничивается фактическим фондом: две смены по 8 часов дают 16 человеко-часов в сутки, из которых 30 % приходится на разбор оповещений — 4,8 человеко-часа. Высвобождается 70 % этой величины: $4,8 \times 0,7 = 3,36$ человеко-часа в сутки, или $3,36 \times 365 \approx 1\,226$ человеко-часов в год.

Физический параметр 3.2 — сокращение времени простоя. Снижение числа уведомлений на инцидент со 132 до 5,5 и сокращение ошибочной адресации с 329 до 13 случаев означают, что авария перестаёт тонуть в потоке и попадает ответственному дежурному сразу. Принимается сокращение времени обнаружения и адресации на 15 минут на инцидент при 6 значимых инцидентах в год: $6 \times 0,25 = 1,5$ часа простоя в год.

Операционные КПЭ. По обязательным ставкам:

- 2.1: $1\,226 \times 2\,000 = 2,45$ млн руб./год;
- 2.2: $1,5 \times 6,25 = 9,38$ млн руб./год.

Суммарный операционный эффект — **11,8 млн руб./год.**

7.6 Затраты

Расходная часть рассчитана по ставке 3 500 руб./ч (таблица 19). Лицензионные отчисления за внешние сервисы отсутствуют: все компоненты, включая инференс языковой модели, размещены в корпоративном контуре.

Таблица 19. Структура затрат

Статья	Расчёт	Сумма	Характер
Разработка и внедрение пилота (6 человек, 8 недель)	$1\,920 \times 3\,500$	6,7 млн руб.	Разовые
Сопровождение и развитие	$360 \times 3\,500$	1,3 млн руб./год	Периодические
Инфраструктура: три узла (см. таблицу 20)	$63\,000 \times 12$	0,76 млн руб./год	Периодические
Лицензии внешних сервисов	—	0	—
Итого разовые		6,7 млн руб.	
Итого периодические		2,0 млн руб./год	

Стоимость инфраструктуры

Стоимость посчитана для аренды вычислительных ресурсов; альтернатива — приобретение сервера с GPU в собственность — переносит основную часть суммы из периодических затрат в разовые и рассматривается как вариант при переходе к промышленной эксплуатации (таблица 20).

Таблица 20. Стоимость инфраструктуры при аренде

Узел	Конфигурация	Стоимость, руб./мес.
Приложения	4 vCPU, 8 ГБ, 50 ГБ	6 000
Данные и шина	8 vCPU, 16 ГБ, 200 ГБ SSD	12 000
ИИ (GPU) [21]	8 vCPU, 32 ГБ, GPU 16 ГБ	45 000
Итого		63 000

Цены являются допущением и подлежат уточнению по действующим договорам; структура расчёта от них не зависит. Доля ИИ-узла составляет 71 % стоимости инфраструктуры — это прямое следствие того, что модель постоянно удерживает видеопамять. При отключённых ИИ-функциях (режим off) годовая стоимость инфраструктуры снижается с 0,76 до 0,22 млн руб.

Альтернативный сценарий: приобретение оборудования

Ориентировочная оценка капитальных затрат при переходе от аренды к собственному оборудованию приведена в таблице 21. Цены заданы по порядку величины исходя из типовой конфигурации серверов корпоративного класса и подлежат уточнению по коммерческим предложениям поставщиков; основной источник неопределённости — стоимость GPU-карты с 16 ГБ видеопамяти.

Таблица 21. Оценка капитальных затрат на оборудование

Узел	Конфигурация	Стоимость, млн руб.
Приложения	4 vCPU, 8 ГБ, 50 ГБ	0,25
Данные и шина	8 vCPU, 16 ГБ, 200 ГБ SSD	0,45
ИИ (GPU)	8 vCPU, 32 ГБ, GPU 16 ГБ	1,66
Итого		2,36

Приобретение не обнуляет периодические расходы на инфраструктуру: остаются размещение в дата-центре, электроэнергия и обслуживание. Эта часть оценена в 15 % стоимости оборудования в год — 0,35 млн руб./год вместо 0,76 млн руб./год при аренде. Итоговые периодические затраты снижаются с 2,0 до 1,65 млн руб./год, а разовые инвестиции возрастают с 6,7 до 9,06 млн руб. (таблица 22); методика дисконтирования — та же, что и в таблице 23.

Таблица 22. Сравнение сценариев аренды и приобретения оборудования

Показатель	Аренда	Приобретение
Разовые инвестиции, млн руб.	6,7	9,06
Периодические затраты, млн руб./год	2,0	1,65
NPV на горизонте 3 года, млн руб.	13,9	12,3
DPP, лет	0,8	1,1

Оба сценария окупаются в пределах горизонта расчёта. Аренда даёт более быстрый DPP за счёт меньших разовых вложений; приобретение — немного ниже итоговый NPV из-за более высокой стартовой инвестиции при том же операционном эффекте. Разница NPV (1,6 млн руб.) меньше точности допущений о ценах на оборудование, поэтому выбор между сценариями определяется доступностью капитальных статей бюджета на этапе пилота, а не экономической целесообразностью.

7.7 Инвестиционные КПЭ

Чистый денежный поток года: $11,8 - 2,0 = 9,8$ млн руб. Дисконтирование по ставке 20 % на горизонте 3 года приведено в таблице 23.

Таблица 23. Финансово-экономическая модель, млн руб.

Показатель	Год 0	Год 1	Год 2	Год 3
Эффект	—	11,8	11,8	11,8
Операционные затраты	—	2,0	2,0	2,0
Инвестиции	6,7	—	—	—
Чистый поток	−6,7	9,8	9,8	9,8
Коэффициент дисконтирования	1,00	0,83	0,69	0,58
Дисконтированный поток	−6,7	8,2	6,8	5,7
Накопленный дисконт. поток	−6,7	1,5	8,3	13,9

- $NPV = 13,9$ млн руб. на горизонте 3 года;
- $PV = 20,7$ млн руб., $PVI = 6,7$ млн руб., $PI = 3,1$;
- $DPP \approx 0,8$ года (около 10 месяцев).

7.8 Чувствительность и риски модели

Модель определяется главным образом эффектом от простоя: при стоимости 6,25 млн руб. за час достаточно предотвратить 1,4 часа простоя в год, чтобы покрыть все затраты первого года (8,7 млн руб.). Отсюда следуют два вывода.

Во-первых, точность оценки числа значимых инцидентов и сокращения времени реакции важнее точности оценки трудозатрат: даже полное обнуление эффекта 2.1 оставляет проект окупаемым ($NPV \approx 8,8$ млн руб.), тогда как ошибка в 1 час простоя меняет годовой эффект на 6,25 млн руб.

Во-вторых, главный экономический риск — не переоценка эффекта, а потеря инцидента: один пропущенный бизнес-критичный инцидент при указанной стоимости простоя способен обнулить годовой эффект целиком. На эталонном корпусе полнота доведена до 8/8, однако это измерение на синтетическом потоке, а не на реальном трафике, поэтому дублирование критичных уведомлений существующим способом сохраняется на время пилота, а полнота остаётся в дереве КПЭ контрольным фактором, а не второстепенной метрикой качества.

Заключение

В ходе работы спроектирована и реализована веб-платформа управления оповещениями для пилотного периметра, отвечающая обоим ограничениям кейса: она не обрабатывает реальные персональные данные и не зависит от домена Active Directory. Роль AD в контуре идентификации выполняет TrueConf: он служит одновременно каналом доставки и провайдером аутентификации.

Получены следующие результаты:

- построена архитектура с детерминированным ядром обработки и необязательным ИИ-контуром, трассированная на требования заказчика (приложение А);
- реализован приём разноформатных событий через сменные адаптеры (Alertmanager, Zabbix, генератор синтетических событий) с приведением к единой открытой схеме;
- реализована корреляция алертов конечным автоматом с решениями дедупликации, подавления и маршрутизации;
- реализованы дашборд и личный кабинет с разделением ролей и настраиваемой ролевой моделью;
- реализована сквозная доставка уведомлений в TrueConf с фиксацией статуса и детерминированным резервным путём;
- реализованы два ИИ-сценария (обогащение алерта и формулировка уведомления) с локальным инференсом, объяснимостью и полным fallback-поведением;
- построен воспроизводимый корпус и измерено качество обработки: снижение числа уведомлений на 95,8 % при полной обнаруживаемости инцидентов — 8 из 8 (таблица 4);
- подготовлено экономическое обоснование с явно выделенными допущениями.

Выявленные ограничения — слияние одновременных инцидентов на общем сервисе и адресация по владельцу сигнализирующего, а не сломавшегося сервиса — измерены количественно и определяют содержание следующего этапа: учёт первопричины и данных сервисного каталога при корреляции и маршрутизации, а также реализация инцидента как самостоятельной сущности с карточкой, подтверждением обработки и каналом обратной связи по решениям ИИ.

Список использованных источников

1. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления : ГОСТ Р : 7.32-2017 / Росстандарт. — Москва : 2017.
2. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания : ГОСТ : 34.601-90 / Госстандарт СССР. — Москва : 1990.
3. Информационные технологии. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Термины и определения : ГОСТ Р : 59853-2021 / Росстандарт. — Москва : 2021.
4. Apache Software Foundation. — Apache Kafka Documentation, 2026. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 2026-08-12).
5. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol : O'Reilly Media, 2017. — ISBN: 978-1-4493-7332-0.
6. Prometheus Authors. — Prometheus and Alertmanager Documentation, 2026. — URL: <https://prometheus.io/docs/> (дата обращения: 2026-08-12).
7. Zabbix LLC. — Zabbix Documentation, 2026. — URL: <https://www.zabbix.com/documentation/current/> (дата обращения: 2026-08-12).
8. Timescale Inc. — TimescaleDB Documentation: Hypertables, Compression and Data Retention, 2026. — URL: <https://docs.timescale.com/> (дата обращения: 2026-08-12).
9. Hardt D. The OAuth 2.0 Authorization Framework. — Электронный ресурс. — 2012. — URL: <https://www.rfc-editor.org/rfc/rfc6749> (дата обращения: 2026-08-12).
10. TrueConf. — TrueConf Server. Руководство администратора: Chatbot API и OAuth 2, 2026. — URL: <https://docs.trueconf.com/server/> (дата обращения: 2026-08-12).
11. Qwen Team. Qwen2.5 Technical Report. — 2024. — arxiv : cs.CL/2412.15115.
12. Docker Inc. — Docker Compose Documentation, 2026. — URL: <https://docs.docker.com/compose/> (дата обращения: 2026-08-12).
13. Site Reliability Engineering: How Google Runs Production Systems / B. Beyer, C. Jones, J. Petoff, N. R. Murphy. — Sebastopol : O'Reilly Media, 2016. — ISBN: 978-1-4919-2912-4.
14. The Site Reliability Workbook: Practical Ways to Implement SRE / B. Beyer, N. R. Murphy, D. K. Rensin и др. — Sebastopol : O'Reilly Media, 2018. — ISBN: 978-1-4920-2950-2.

15. OWASP Foundation. OWASP Top 10 for Large Language Model Applications, version 2025.— Электронный ресурс.— 2025.— URL: <https://owasp.org/www-project-top-10-for-large-language-model-applications/> (дата обращения: 2026-08-12).
16. National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0).— Электронный ресурс.— 2023.— URL: <https://www.nist.gov/itl/ai-risk-management-framework> (дата обращения: 2026-08-12).
17. OWASP Foundation. OWASP Application Security Verification Standard (ASVS).— Электронный ресурс.— 2024.— URL: <https://owasp.org/www-project-application-security-verification-standard/> (дата обращения: 2026-08-12).
18. О безопасности критической информационной инфраструктуры Российской Федерации : Федеральный закон : 187-ФЗ / Российская Федерация. — Москва : 2017.
19. Об утверждении Требований по обеспечению безопасности значимых объектов критической информационной инфраструктуры Российской Федерации : Приказ : 239 / ФСТЭК России. — Москва : 2017. — В ред. от 20.02.2020.
20. Информационные технологии. Методы и средства обеспечения безопасности. Системы менеджмента информационной безопасности. Требования : ГОСТ Р ИСО/МЭК : 27001-2021 / Росстандарт. — Москва : 2021.
21. vLLM Project. — vLLM Documentation: Easy, Fast, and Cheap LLM Serving, 2026.— URL: <https://docs.vllm.ai/> (дата обращения: 2026-08-12).
22. Newman S. Building Microservices: Designing Fine-Grained Systems.— 2nd изд.— Sebastopol : O'Reilly Media, 2021.— ISBN: 978-1-4920-3402-5.
23. Limoncelli T. A., Chalup S. R., Hogan C. J. The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems. — Boston : Addison-Wesley, 2017.— ISBN: 978-0-321-94318-7.
24. Efficient Memory Management for Large Language Model Serving with PagedAttention / W. Kwon, Z. Li, S. Zhuang и др. // Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23). — New York : ACM, 2023. — С. 611–626. — arxiv : cs.LG/2309.06180.
25. Ramírez S.— FastAPI Documentation, 2026.— URL: <https://fastapi.tiangolo.com/> (дата обращения: 2026-08-12).

Приложение А

Матрица трассировки требований на архитектурные решения

Матрица связывает требования заказчика с архитектурными решениями и критериями их проверки. Идентификаторы требований сохранены в форме, предоставленной заказчиком. Решения описываются как компоненты и их обязанности: конкретные продукты не фиксируются там, где выбор не сделан.

Таблица А1. Матрица трассировки требований на архитектурные решения

ID	Архитектурное решение	Требования-основания	Критерий приёмки
AP-01	Ввести единый контур приёма событий: API gateway/WAF, push webhook/HTTP и индивидуальные конфигурируемые адаптеры для каждой системы мониторинга. Адаптеры приводят события к единой внутренней схеме.	БТ-1; ФТ1.1, ФТ1.2, ФТ1.4; НФТ1.4; НФТ4.1, НФТ4.11, НФТ4.13	Новый источник подключается маппингом полей без изменения кода; событие принимается по защищённому каналу и нормализуется; отказ адаптера не приводит к потере накопленных событий.
AP-02	Использовать устойчивую асинхронную шину событий с очередями, consumer groups, приоритизацией, backpressure и отдельной очередью ошибок (DLQ).	БТ-1; НФТ1.4; НФТ4.5, НФТ4.9, НФТ4.12; ФТ6.3; НФТ7.3	При недоступности потребителя события сохраняются и обрабатываются после восстановления; критичные алерты имеют приоритет; RPO = 0 и RTO приёма не более 3 минут.
AP-03	Реализовать детерминированное ядро обработки: настраиваемые правила критичности, типизации, дедубликации и корреляции. Это гарантированный путь обработки, независимый от LLM.	БТ-2; ПТ2.1; ФТ2.1, ФТ2.2, ФТ2.3; БТ-6; ФТ6.2, ФТ6.3; НФТ6.1; БТ-7; ПТ7.2, ПТ7.3; НФТ7.2	Правила меняются без переработки истории; при отключённом AI события классифицируются, коррелируются, маршрутизируются и доставляются.
AP-04	Хранить классификации, правила, пороги уверенности, режимы автоматизации и fallback-политику как конфигурацию с версионированием, а не как жёстко заданный код.	ПТ2.1; НФТ2.3; ФТ6.1; ПТ7.1, ПТ7.2; ФТ7.6; НФТ7.1	Новая категория добавляется без пересчёта истории; для каждой AI-функции задаются режим, confidence threshold и fallback; версия политики видна в аудите.
AP-05	Предоставить централизованный web frontend: дашборд администратора и личный кабинет пользователя с текущими авариями, поиском, историей, аудитом, подписками и политиками.	ПТ2.2; ПТ3.1; НФТ2.1; НФТ3.1; ФТ7.7	Новое событие видно на панели не позднее 15 секунд; изменение подписки в UI получает отклик не позднее 3 секунд; пользователь может подтвердить, отклонить или перепреопределить AI-решение.
AP-06	Выделить компонент маршрутизации и уведомлений: подписки по источнику/типу/критичности/активу, шаблоны уведомлений, обязательная интеграция с TrueConf, опциональные каналы, статус доставки и fallback.	БТ-3; ФТ3.1, ФТ3.2, ФТ3.4, ФТ3.5, ФТ3.7; НФТ3.2, НФТ3.3, НФТ3.4; НФТ4.14	Изменённая подписка применяется к новым событиям не позднее 5 секунд; при недоступности основного канала fallback срабатывает не позднее 1 минуты; доставка каждого уведомления имеет зафиксированный статус.

Продолжение таблицы А1

ID	Архитектурное решение	Требования-основания	Критерий приёмки
AP-07	Использовать OAuth 2 через TrueConf как основной путь аутентификации; поддерживать локальные учётные записи. AD использовать только как номинальный справочник для статуса/атрибутов пользователя, без зависимости от реального AD на пилоте.	БТ-4; ФТ4.1, ФТ4.2, ФТ4.3; НФТ4.3, НФТ4.4	Пользователь аутентифицируется через TrueConf или локальную учётную запись; права назначаются ролями и группами; подписки деактивируются при сигнале о кадровом изменении; в БД нет ФИО, телефонов и иных ПДн.
AP-08	Хранить в TimescaleDB (PostgreSQL-совместимая БД) события, алерты и статусы доставки в hypertables с настраиваемыми retention и сжатием; хранить состояние инцидентов, подписки, политики, аудит и только технические, псевдонимизированные идентификаторы пользователей как реляционные записи.	ФТ1.6; НФТ4.2, НФТ4.4; НФТ7.1	История за заданный временной диапазон доступна в пределах retention; hypertables сжимаются по политике; журнал действий хранится не менее года; журнал AI-решений содержит модель, версию, confidence, входные признаки и результат.
AP-09	Использовать CMDb/сервисный каталог как источник контекста: объекты, сервисы, владельцы, команды, on-call и зоны ответственности. Кэшировать горячий контекст, чтобы не сделать CMDb bottleneck.	ФТ3.1; ФТ7.3, ФТ7.4; НФТ4.12	Маршрут определяется по активу и владельцу; при низкой уверенности алерт направляется в triage-очередь; недоступность или медленность CMDb не блокирует приём критичных алертов.
AP-10	Реализовать отдельный необязательный AI-контур через асинхронный worker и контракт сообщений. Поддерживать функции: семантическая дедупликация, фильтрация шума/карантин, приоритизация, маршрутизация, саммаризация, декомпозиция и объяснение решений.	БТ-2; ФТ2.2, ФТ2.3, ФТ2.4; БТ-7; ФТ7.1-ФТ7.7; НФТ7.1-НФТ7.4	AI-решение содержит explanation, confidence, признаки, альтернативы и версию модели; critical/high не скрываются, не подавляются и не понижаются автоматически; при низкой уверенности применяется triage/fallback.
AP-11	Размещать LLM-инференс и дообучение только в корпоративном контуре. Выделить внутренний LLM gateway/балансировщик и управляемый пул inference; исключить внешние облачные API для модели.	НФТ2.2; НФТ7.4	Сетевые политики исключают исходящий доступ AI-контура к внешним LLM API; инвентаризация показывает, что модели, inference и обучение находятся в контуре компании.
AP-12	Не делать AI/LLM синхронной точкой отказа основного потока. При сбое AI применять детерминированные правила и зафиксированное fallback-поведение; использовать feature flags и режимы off/shadow/suggest/assisted auto/full auto.	БТ-6; ФТ6.1-ФТ6.3; НФТ6.1; ПТ7.1-ПТ7.3; НФТ7.2, НФТ7.3	Отключение AI worker или inference не задерживает приём, регистрацию и доставку critical-алерта; режим AI переключается без развёртывания ядра.
AP-13	Построить наблюдаемость и аудит как сквозную функцию: структурированные логи, метрики, трассировка, Kafka lag, SLO, аудит пользователей, изменений БД, решений AI и статусов доставки. Исключать секреты и чувствительные данные из логов.	НФТ4.2, НФТ4.8, НФТ4.10; НФТ7.1; НФТ3.4; НФТ2.1	Можно отследить путь события от приёма до доставки; можно доказать, кто изменил правило/подписку; автоматическая проверка не обнаруживает секретов или чувствительных данных в логах.

Продолжение таблицы А1

ID	Архитектурное решение	Требования-основания	Критерий приёмки
AP-14	Применить defense in depth: TLS 1.2+, сегментация сети, закрытые БД/очереди, минимальные привилегии, индивидуальные service accounts, секреты в vault, резервное копирование и регулярные тесты восстановления.	НФТ4.1, НФТ4.5-НФТ4.10	Внешние интеграции используют TLS 1.2+; доступ к БД и очередям ограничен сетевыми правилами и сервисными учётными записями; восстановление из backup регулярно проверяется.
AP-15	Защищать входной и исходящий трафик: WAF/API gateway, rate limiting для API/аутентификации, квоты на уведомления, защита от alert storm и отдельная политика critical-приоритета.	НФТ4.11-НФТ4.14; НФТ1.4	Аномальный HTTP-трафик ограничивается на периметре; лавина событий не обрушает обработку; квоты предотвращают массовую рассылку, не блокируя допустимые critical-эскалации.
AP-16	Вести пилот через поэтапное подключение источников, feature flags, критерии успеха, документированный rollback и временное дублирование критичных уведомлений.	БТ-5; ПТ5.1-ПТ5.3; НФТ5.1; ФТ6.1	Каждый этап имеет список источников, метрики успеха, владельца и rollback-план; критичные уведомления можно временно дублировать; откат укладывается в согласованное значение X минут.
AP-17	Включить симулятор событий как равноправный тестовый источник, использующий те же схемы, адаптеры и поток обработки, что и реальные системы мониторинга.	БЦ; БТ-1; ФТ1.1, ФТ1.4; НФТ1.4; БТ-5; ПТ5.1, ПТ5.2	Без доступа к инфраструктуре компании симулятор создаёт сценарии normal/critical/storm/failure; они проходят полный путь до dashboard и уведомления и используются для проверки rollout/rollback.

Приложение Б

Детальная схема архитектуры

Схема объединяет все представления главы 1 и приводится как справочный материал: она перегружена для чтения по ходу текста, но полезна как единая карта компонентов и связей.

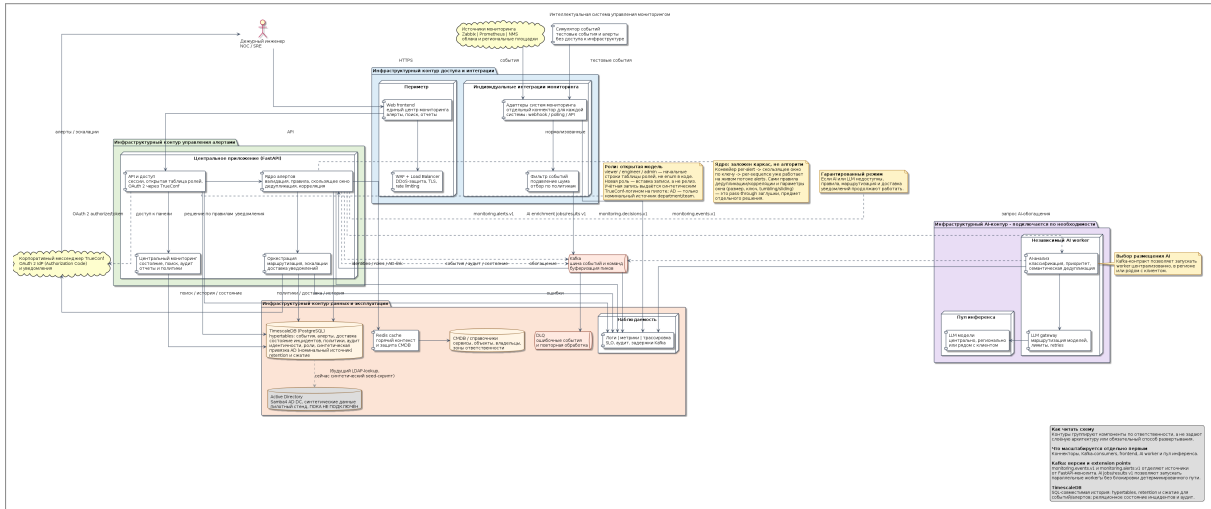


Рисунок Б.1. Детальная схема архитектуры решения

Приложение В

Схема данных

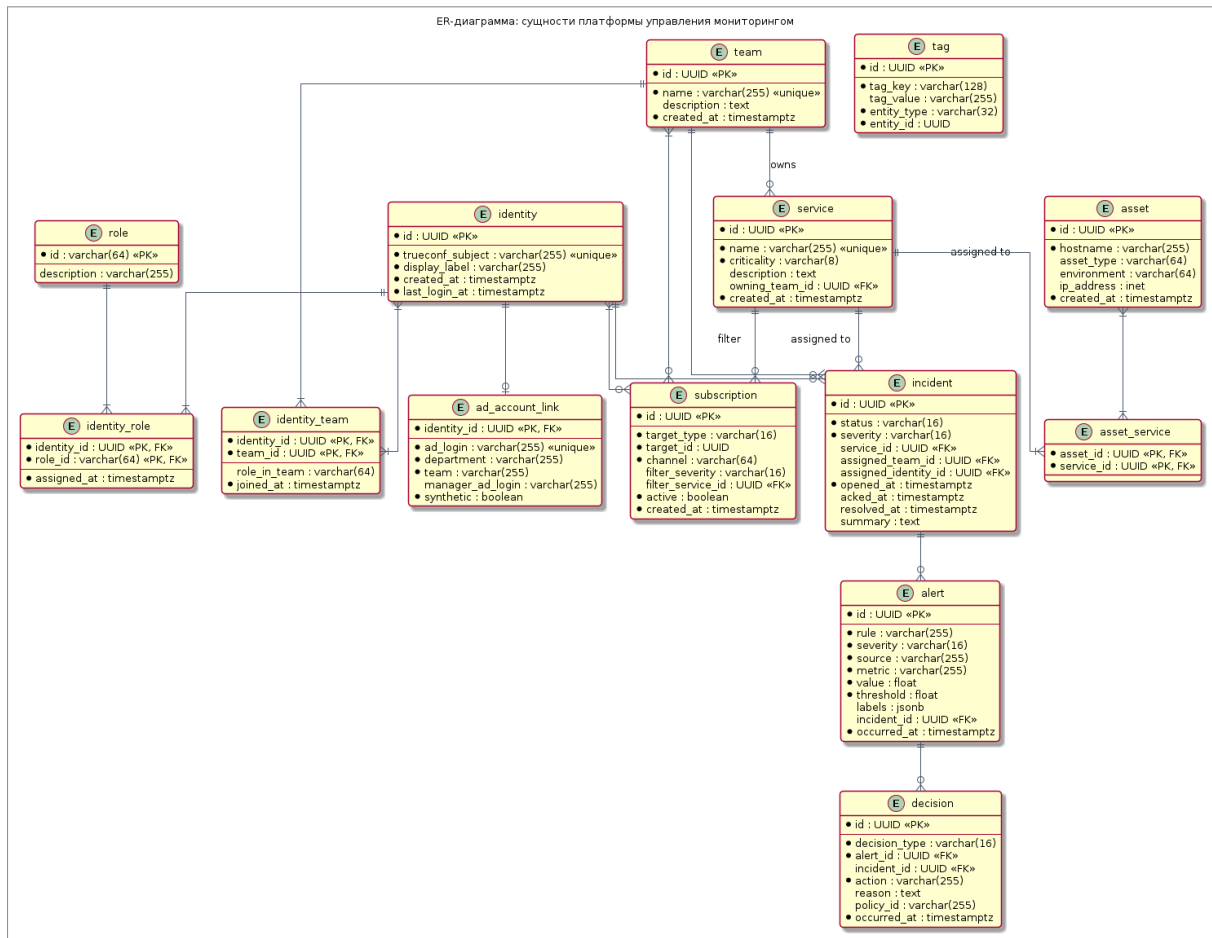


Рисунок В.1. Схема сущностей и связей